Dataset Distillation via a Noise-Unconstrained Generative Model

Jingxuan Zhang, Lei Dai, *Member, IEEE*, Fei Ye, Zhihua Chen, *Member, IEEE*, Ping Li, *Member, IEEE*, Xiaokang Yang, *Fellow, IEEE*, and Bin Sheng, *Senior Member, IEEE*

***Abstract*—Dataset distillation (DD) aims to synthesize a more compact dataset than the original one and models trained on it are expected to have the same generalization capabilities as on the original dataset. Previous work via a generative model (GM) faces several limitations. First, GM struggles to generate representative samples due to a lack of constraints. Second, it overlooks the relationships between generated samples, limiting its effectiveness. In this paper, a new noise-unconstrained GM-based DD framework is proposed. In the distillation stage, an adaptive matching coefficient is introduced to align generated images with representative class elements and the MiniMax loss function is extended to reduce the optimization difficulty. In the deployment stage, features among each generative image are ensembled by gradient-matching based DD. Theoretical analysis based on McDiarmid's inequality demonstrates that the proposed components can reduce the generalization error of the original baseline method. We also provide insights into the potential of generated images as an effective proxy dataset for DD. For example, on the ImageWoof dataset with 50 distilled images per class using a 6-layer ConvNet for evaluation, generated images outperform 25%, 50%, and 75% original images by 8.4%, 6.3%, and 8.3% in distillation performance. Our method effectively handles both low- and high-resolution datasets, with experiments on 11 benchmarks demonstrating its efficacy.**

***Index Terms*—Dataset distillation, generative adversarial network, diffusion model, image classification.**

I. Introduction

DATASET distillation (DD), a promising research direction [1], aims to obtain an extremely compact and informative dataset given the redundant one. Models training on the distilled dataset are expected to achieve almost the same test set accuracy as on the original one. DD can significantly reduce the storage of large datasets, and alleviate the computational burden, environmental impact, and privacy issues while training the model on the original large datasets, such as ImageNet1K [2]. DD, as an extension of the coreset which selects a subset of the original dataset, offers greater flexibility and can fit the distribution of training samples effectively.

Nowadays, there exist numerous DD methods [1] and most of them optimized the dataset in the pixel space. For instance, gradient matching [3], [4], [5], a popular framework, aims to ensure that the model parameters trained on large and small datasets are as similar as possible. However, these methods have poor redeployment efficiency and suffer from poor cross-architecture performance. Another line of work resorted to a generative model (GM) to construct that small dataset. They either optimized the input noise [6] or the generative model [7] or both of them [8]. Unlike traditional generative models focused on creating visually appealing images, in DD, models trained on a limited set of generated results can achieve excellent classification performance. This approach benefits from the ability of the generative model to produce realistic images, ensuring robust performance across different architectures. Additionally, varying the Gaussian noise samples allows for generating the required number of distilled images. However, challenges lie in that, firstly, the limitation of the GM only ensures that the generated images look realistic, but there is no restriction on whether the generated images are representative of the original dataset [7]. Secondly, previous work just generated the same number of images as the final requirement of distilled images, failing to tap the potential of GM. Thirdly, previous work did not reveal the differences between distilling original images and distilling generated images.

To address these challenges, we design an adaptive matching coefficient that encourages the GM to generate images whose features are closer to the cluster centers of each class, thereby making the generated images more representative. The original MiniMax losses are extended to enforce constraints on additional generated images that exhibit large deviations from representative real samples or high similarity to generated images from neighboring iterations. Furthermore, to fully exploit the generative capacity of the GM, images are generated more than the required IPC, then we treat those generated images as a proxy dataset and perform a secondary distillation to obtain the

Received 26 February 2025; revised 26 February 2026; accepted 25 April 2026. Date of publication 6 May 2026; date of current version 6 August 2026. This work was supported by the National Natural Science Foundation of China under Grant T2525004, Grant 62272164, Grant 62572188, and Grant 62306113, in part by Shenzhen Medical Research Fund under Grant E250200613, and in part by the State Key Laboratory of Industrial Control Technology, China under Grant ICT2026A25. Recommended for acceptance by V. Murino. *(Corresponding authors: Zhihua Chen; Bin Sheng.)*

Jingxuan Zhang, Lei Dai, and Zhihua Chen are with the Department of Computer Science and Engineering, East China University of Science and Technology, Shanghai 200237, China, and also with the State Key Laboratory of Industrial Control Technology, East China University of Science and Technology, Shanghai 200237, China (e-mail: y21220033@mail.ecust.edu.cn; dailei@ecust.edu.cn; czh@ecust.edu.cn).

Fei Ye is with the School of Information and Software Engineering, University of Electronic Science and Technology of China, Chengdu 610054, China (e-mail: feiye@uestc.edu.cn).

Ping Li is with the Department of Computing, Hong Kong Polytechnic University, Hong Kong (e-mail: p.li@polyu.edu.hk).

Xiaokang Yang and Bin Sheng are with the School of Computer Science, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: xkyang@sjtu.edu.cn; shengbin@sjtu.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TPAMI.2026.3690778, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2026.3690778

final results. Theoretically, the former reduces the variance of the generated image features, while the latter lowers the training loss by distilling a proxy dataset containing more diverse samples. The combination of these two strategies, based on McDiarmid's inequality [9], leads to a smaller generalization error than the baseline method. Finally, compared with the original dataset, the smaller variations among the generated images yield smoother distillation training, further demonstrating the advantages of using generated images as a proxy dataset.

In this paper, we adapt our method to two different generative models, GANs and diffusion models, to handle low- and high-resolution datasets respectively. Note that for high-resolution datasets, all distillation processes including pre-training the classification model and gradient matching are conducted in the latent space to ensure computational efficiency. In addition, cross-architecture experiments further validate the effectiveness of the proposed approach. Besides, we provide relevant theoretical explanations based on McDiarmid's inequality.

In summary, our work makes three main contributions:

- While fine-tuning the generative model, an adaptive coefficient for the matching function is proposed for the small-resolution dataset. The coefficient is proportional to the distance in the feature space between the generated images and the clustering centers of each class. For large-resolution datasets, the MiniMax loss is extended to cover additional generated images that either significantly deviate from representative real samples or are close to generated images from adjacent iterations.
- The generative model is fully utilized by first generating an excess of images and subsequently condensing them to obtain the final distilled dataset. This method applies to both distilling small-resolution datasets generated using a generative adversarial network in the image space and distilling large datasets generated by a diffusion model in the latent space. Theoretical analysis leveraging McDiarmid's inequality shows that our approach can achieve a reduced generalization error.
- We explored the differences between distilling with a subset of the original dataset and distilling with generated images. The similarity between generated images within the same class ensures that the distillation results from generated images are closer to the initialization with real images, leading to a more stable distillation process and better cross-architecture performance. Such a study offers insights into the complex characteristics of generated images and their potential as a proxy dataset.

II. Related Work

In this context, we will examine the most pertinent studies on generative models (GMs), coreset selection, and DD.

A. Generative Models

GM initially aims to output realistic images and has been widely used in image-to-image translation, image super-resolution, and conditional image generation [10]. The core technology is GAN and the diffusion model.

GAN comprises a generator and a discriminator. The generator aims to produce images that appear authentically realistic to deceive the discriminator, while the objective of the discriminator is to discern whether the generated images are genuine. One of the challenges of GANs is the poor training stability. Gulrajani et al. [11] proposed an improved WGAN by employing the gradient penalty to regularize the gradients of the discriminator to facilitate the convergence of the model. Miyato et al. [12] constrained the Lipschitz norm of the discriminator for stable training. Brock et al. [13] proposed BigGAN, which enlarged the parameters 2–4 times and introduced a truncation trick to the noise to improve the sample quality.

Diffusion model is another branch of GM and is becoming more and more popular nowadays. Unlike GAN, its training process is more stable and suitable for training on large-scale datasets. It contains diffusion and denoising processes [14]. In the diffusion process, Gaussian noise with pre-set mean and variance is gradually added to the clean image while a U-Net architecture is trained to estimate the noise present in the denoising procedure. Rombach et al. [15] proposed stable diffusion to reduce the training cost of the diffusion model. Firstly, a VAE encoder maps the images into the latent space. Subsequently, diffusion and denoising operations are performed in the latent space. Finally, the latent vectors are mapped back to the image space through a VAE decoder. Song et al. [16] proposed denoising diffusion implicit models (DDIM) by building a non-Markov process to accelerate the sampling phrase. Peebles et al. [17] replaced the U-Net architecture in the denoising network with a Transformer model. Xie et al. [18] investigated various methods for fine-tuning the pre-trained diffusion model to adapt to the new domains. They only trained the bias terms and a few scaling factors while freezing most parameters. Nevertheless, these methods do not ensure that the generated images are beneficial for the training of the model.

B. Coreset Selection

Coreset selection is a natural idea of reducing the size of training datasets by picking the most representative samples. Sener et al. [19] proposed a K-center algorithm. Its main idea is to choose the center points whose maximum distance between the data and its nearest centroid is minimized. Welling et al. [20] selected samples adhered to the constraints of moments. Currently, algorithms based on coreset selection exhibit inferior performance when the number of distilled images per class is low compared to DD methods.

C. Dataset Distillation

DD can be generally classified as methods directly optimizing image pixels and those based on GMs.

Optimizing image pixels directly is one of the natural ideas to do DD. These methods can be classified as performance, gradient, and distribution matching based algorithms. Wang et al. [21], which was the first DD work, employed a meta-learning framework. To improve the computational cost in [21], Nguyen et al. [22] and Loo et al. [23] used the Neural Tangent Kernel (NTK) theory to compute the closed-form solution of

the inner loop. Feng et al. [24] randomly truncated one of the trajectories in the inner-level optimization to accelerate the DD. However, the large GPU memory consumption prevents us from running the process on a single 24 GB GPU, even under default conditions with CIFAR10 at IPC=10, a dataset with relatively small resolution. Gradient matching based methods made a stronger assumption that the parameters of a model trained on the original and distilled datasets are as similar as possible. Zhao et al. [3] aligned the gradients during the training of the same model on the two datasets. To avoid the accumulated errors in [3], Cazenavette et al. [4] aligned the loss function during a long training period. To develop more efficient DD algorithms, Zhao et al. [25] reduced the disparity in feature distribution between the original and condensed datasets, eliminating the computational bi-level optimization. Sajedi et al. [26] improved [25] by matching the most informative feature maps through the attention mechanism. Zhao et al. [27] proposed a data augmentation method, a data structure to sample the pre-trained models, and added a class-aware regularization term to the matching function. Deng et al. [28] improved [25] by adding a class-centric constraint and a covariance alignment constraint. Zhang et al. [29] matched the gradients of the early pre-trained checkpoints, which could produce significant and diverse gradients. Cui et al. [30] improved [4] via a memory-releasing trick. Yin et al. [31] matched batch normalization layers via a fully pre-trained model. Shao et al. [32], building on [31], employed data densification to ensure that each class of data remains linearly independent. Additionally, they utilized generalized statistical matching and generalized backbone matching to ensure the distilled data contains rich and diverse information. Sun et al. [33] chose part of the images with low training loss and then combined them. Additionally, researchers have proposed numerous plug-and-play methods. Liu et al. [34] proposed a more efficient initialization method of the distilled dataset to accelerate the convergence speed. Kim et al. [5] reduced the size and increased the number of images to enlarge the information in the distilled dataset. Xu et al. [35] proposed a method of pruning the original dataset for distillation without adopting a generative model to conduct DD. Chen et al. [36] proposed a progressive DD method which in each stage, a new set of images is distilled while keeping the previously distilled images fixed. All sets of distilled images are then used to train a final classification model. Similarly, our method generates different sets of images in each training iteration. However, differences lie in that first, initialization of each distilled image for [36] is from the original dataset while our method is from a generative model. Second, we only use the distilled images for the current stage to train the classification model rather than using all the generative images from the beginning to the current iteration as [36]. To mitigate the forgetting problem, we utilize an EMA technique which does not take extra training time at all. Third, the motivation of those two papers is different. [36] aims to capture different training dynamics in the different stages of training while our method aims to unleash the power of a generative model. Duan et al. [37] did DD on the latent space to extend their method to the high-resolution dataset. We also opt to perform distillation in the latent space on high-resolution datasets. The distinction between our method and [37] lies in the fact that our original dataset is generated using a generative model. Secondly, the size of our original dataset is adjustable. Therefore, our method only requires storing a generative model, whereas [37] needs to store the entire original dataset. When the original dataset requires a large space, [37] demands more storage space.

Generative-based methods: Unlike the previous category of methods, those based on GMs shift the optimization target from image pixels to the model, generating a dataset through a GM. Cazenavette et al. [6] trained the input noise of a GAN and generated a distilled dataset based on a pre-trained GAN. Wang et al. [7] proposed DiM, which decomposed the whole DD into two stages, that is, distillation and deployment stages. In the distillation stage, DiM trained GAN via the logit matching but without constraints on the input noise. DiM generated a specified number of images in the deployment stage as the final distillation results. Li et al. [38] enhanced DiM by normalizing the logits and employing a Kullback-Leibler divergence-based matching function for dataset alignment. Zhang et al. [8], following DiM, simultaneously optimized the input noise and GAN. Gu et al. [39] used the generative prior of a diffusion model and proposed two new MiniMax-based loss functions to improve the representativeness and diversity of diffusion model. Although empirical results demonstrate excellent performance in cross-architecture scenarios, and some of them [7], [39] are easily redeployed (no need to rerun all programs when the quantity of distilled images per class changes), they do not fully unleash the power of GM since GM can produce far more than 50 (typically the largest number in the field of DD) images per class. However, our method maximizes the capabilities of the generative model and proceeds with compression. Such work explores the integration of generative-based DD with traditional DD which is one of the future research directions outlined in the most recent survey paper [40].

III. Methodology

A. Problem Definition

Let's consider the original dataset denoted as $\mathcal{T} = \bigcup_{i=1}^{n} (\boldsymbol{x_i}, y_i)$, where n represents the number of images, $\boldsymbol{x_i} \in \mathbb{R}^{3\times w\times h}$ represents the i^{th} original image, $y_i \in \mathbb{R}^c$ is the i^{th} original ground truth category, w, h represent the width, height of the images, respectively and c indicates the number of classes. The goal of dataset distillation is to obtain another dataset $\mathcal{S} = \bigcup_{i=1}^{m} (\boldsymbol{x'_i}, y'_i)$, whose size is much smaller than the original one: $m \ll n$. $\boldsymbol{x'_i}$ and y'_i denote the i^{th} synthetic image-label pairs. In subsequent notation, we will denote the set of original images and the set of distilled images as $\boldsymbol{x}$ and $\boldsymbol{x'}$, respectively. y and y' are defined in the same manner. $\tilde{\boldsymbol{x}}_{\boldsymbol{i}} = (\boldsymbol{x'_i}, y'_i) \in \mathcal{S}$. Note that $\boldsymbol{x'_i} \in \mathbb{R}^{3\times w\times h}$, $y'_i \in \mathbb{R}^c$ and the number of classes in the original and distilled datasets are the same in this paper. $\mathcal{S}$ needs to satisfy that any two models, namely deep neural networks, sharing the same architecture denoted as $f_{\theta^\mathcal{T}}$ and $f_{\theta^\mathcal{S}}$ with parameters $\theta^\mathcal{T}$ and $\theta^\mathcal{S}$, trained on datasets $\mathcal{T}$ and $\mathcal{S}$ respectively, will demonstrate nearly identical performance on

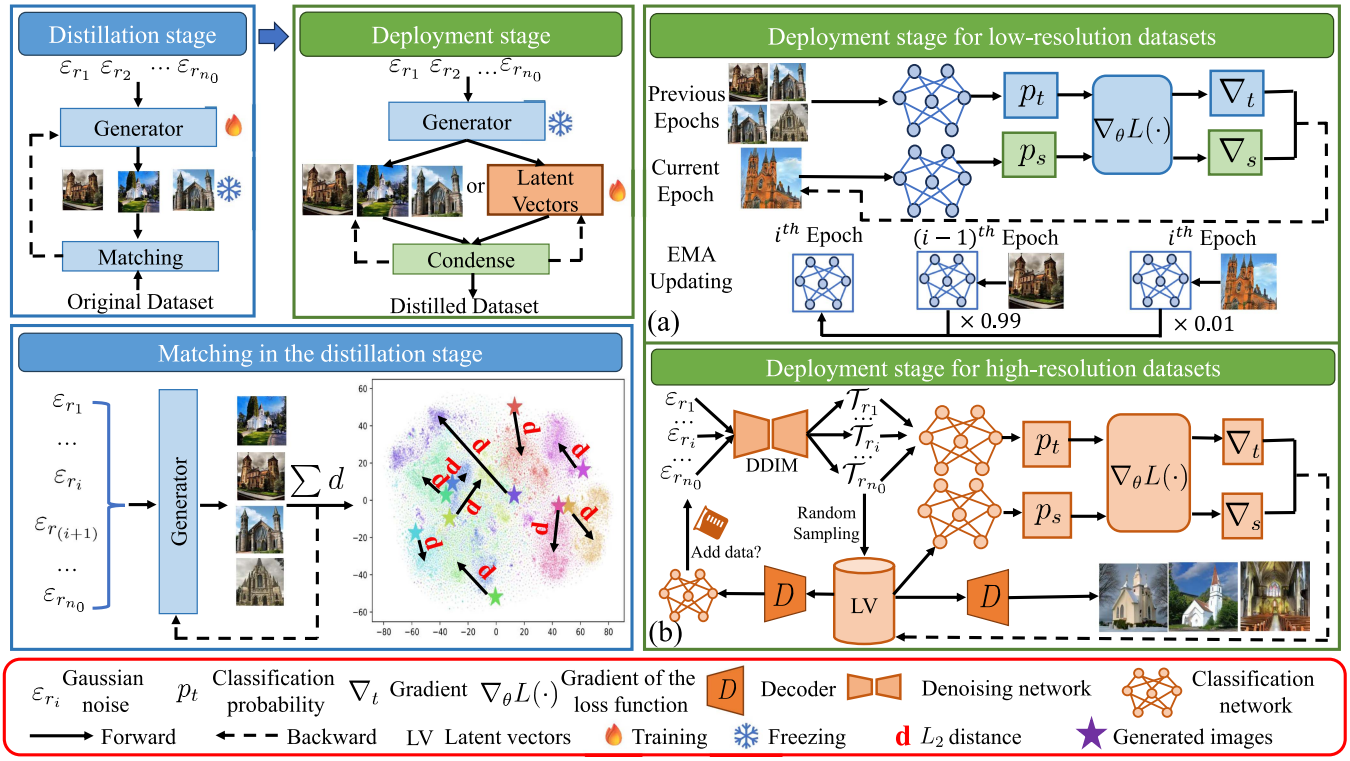


Fig. 1. Overall structure of our proposed dataset distillation algorithm. The overall framework consists of a distillation stage and a deployment stage, which are shown in the top-left corner. In the distillation stage (shown in the bottom-left corner), we optimize the generative model based on the distance between the features of generated and real images. In the deployment stage (shown in the right part), more than the required number of images per class are sampled either from the previous epochs directly (a) or from sampling offline (b), and then, they are condensed to the final distilled dataset either in the image space (a) or in the latent space (b).

an unseen test dataset:

$$\mathbb{E}_{(\boldsymbol{x},y)\sim P_{test}}\left[\ell\left(f_{\theta^{\mathcal{T}}}(\boldsymbol{x}),y\right)\right]\simeq\mathbb{E}_{(\boldsymbol{x},y)\sim P_{test}}\left[\ell\left(f_{\theta^{\mathcal{S}}}(\boldsymbol{x}),y\right)\right], \tag{1}$$

where ℓ is the loss function (typically a cross-entropy loss for a classification task) and P_{test} is the distribution of the test set. Main notations throughout this paper are listed in Table I for convenience.

B. Overall Framework

The comprehensive framework of our proposed method illustrated in Fig. 1 contains two parts: the distillation and deployment stages. The distillation stage includes training GM from scratch or fine-tuning it by adding extra loss to make the results of GM more representative. The deployment stage includes fine-tuning the results in the previous stage and obtaining the final distilled dataset. In the deployment stage, the computational workload and operation speed are significantly smaller than redistilling on the image space. The significant advantage of this framework compared to those directly optimizing on the pixel space is that regardless of the number of distilled images for each class, they will share the same distillation phase. Therefore, the redeployment of our method is more efficient than those that directly optimize the pixels.

C. Distillation Stage

The overall goal of the distillation stage is to train a GM with some extra loss to boost the performance of DD. This additional loss is typically derived from the distance between generated and real images. Specifically, we detail its design for GAN models on low-resolution datasets and diffusion models trained on high-resolution datasets as follows.

1) Low-resolution datasets: We designed a novel generative loss function for the DD of small datasets: the greater the difference between the characteristics of the generated images and the centers of each class in the dataset, the higher the weight of the matching loss.

Suppose $C=\{C_1,C_2,\ldots,C_k\}$ are k clusters for a certain class. We first use the K-means algorithm to solve C:

$$C=\underset{C}{\operatorname{argmin}}\sum_{i=1}^{k}\sum_{\boldsymbol{x}\in C_i}\left\|\boldsymbol{x}-\frac{1}{|C_i|}\sum_{\boldsymbol{x}\in C_i}\boldsymbol{x}\right\|_2^2, \tag{2}$$

where $|\cdot|$ is the size of the set. Then, we pre-train p neural networks with different architectures and layers l_i: $g_1^{\{l_1\}},g_2^{\{l_2\}},\ldots,g_p^{\{l_p\}}$. In the experiment, we pre-train a total of five different network architectures in practice: a 3-layer ConvNet, a 10-layer ResNet, an 18-layer ResNet, a 10-layer ResNetAP, and an 18-layer ResNetAP. Utilizing different network structures can result in more diverse and enriched feature

TABLE I
MEANING OF THE NOTATIONS THROUGHOUT THE PAPER

Notation	Meaning
$\mathcal{T}$ and $\mathcal{S}$	Original dataset and distilled dataset.
$\mathcal{T}_0$ and $\mathcal{S}_0$	Original latent vectors and distilled latent vectors.
n, m, and n_0	The number of images in the original and distilled dataset and the number of initial latent vectors per class.
$\boldsymbol{x}$, $\boldsymbol{x}'$, $\boldsymbol{x_i}$ and $\boldsymbol{x}_i'$	Images in the original and distilled dataset. The i^{th} image in the original and distilled dataset.
y, y', y_i and y_i'	Class labels in the original and distilled dataset. The i^{th} class label in the original and distilled dataset.
$\tilde{\boldsymbol{x}}$	Image-label pair in the synthetic dataset.
IPC, c, w, h	The number of images per class in the distilled dataset, the number of categories in the original dataset, the width and the height of the image in the original dataset.
$\ell(A, B)$	The cross-entropy loss between the predicted results A and the ground truth labels B.
C, C_i and k	All clusters within a certain category. The i^{th} cluster in C and the number of cluster in C.
d	Feature distance between the cluster and the generated images.
$g_i^{\{l_i\}}$ and $f_{\theta\mathcal{T}}$	The i^{th} architecture with the size of layer l_i for extracting the features of clusters, a classification model for gradient matching trained on dataset $\mathcal{T}$ with parameters θ.
h^i	The ensemble of the i^{th} and $(i-1)^{th}$ iteration of f by exponential moving average (EMA).
p and l	The number of architectures $g_i^{\{l_i\}}$ and the minimum layer of those architectures: $\bigcup_{i=1}^{p} g_i^{\{l_i\}}$
$\mathcal{G}_{\theta_1}$, $\mathcal{D}_{\theta_2}$ and D	Generator with parameters θ_1 and discriminator with parameters θ_2 in the generative adversarial network and a pre-trained VAE decoder.
P_{test} and $P(r)$	Distribution of the test set and original dataset of class r.
$P_{\mathcal{G}_{\theta_1}}(\varepsilon, r)$	Distribution of the generated dataset of class r by a generator $\mathcal{G}$ with parameters θ_1 and input Gaussian noise ε.
λ_1, λ_2 and α	Weight of the adaptive coefficient, gradient matching and the weight decay ratio in the EMA.
K	Hyper-parameter used in the extended MiniMax loss function to compute the distance to other real or generated images.
$\bar{w}$ and $CurW$	Pre-set sliding window size and a variable to record the current window size.
$\mathcal{T}_0$ and $\mathcal{T}_{cache}$	The original images generated by the generator in the current round, along with the images within a sliding window of size $\bar{w}$.
$\mathcal{T}_{cache}.append(\mathcal{T}_0)$	Append $\mathcal{T}_0$ to the end of $\mathcal{T}_{cache}$.
$\mathcal{T}_{cache}.pop(0)$	Pop the first element added to $\mathcal{T}_{cache}$.
M, $x_{(s)}$ and β_t	The total number of sampling steps, the s^{th} step of sampling images, and the pre-set t^{th} variance schedule in the diffusion denoising implicit model.
ε and $\varepsilon_{\theta_3}(\boldsymbol{x_{(t)}}, t)$	Gaussian noise sampled from $\boldsymbol{N(0,1)}$ and a denoising network whose input is the t^{th} sampled image $\boldsymbol{x_{(t)}}$ and step t with parameters θ_3.
$\eta_{\mathcal{S}}$ and η_θ	Learning rate for updating the synthetic dataset/latent vectors and learning rate for updating the classification model.
SM, $\nabla_\theta \ell$ and Δ	The softmax operator, the gradient of ℓ with respect to θ and the absolute value of the difference.

extraction. For each cluster and each architecture, we extract the features of these clusters $\bigcup_{i=1}^{p}\{\bigcup_{j=1}^{k}\{g_i(C_j)\}\}$ in the different layers and save them in the disk.

For the matching stage, given a conditional generator $\mathcal{G}_{\theta_1}$ and a certain class index r, the feature distance d between the generated result and the cluster is calculated as follows:

$$d = \sum_{j=1}^{l} \left\| g_i^{\{j\}}(\mathcal{G}_{\theta_1}(\varepsilon, r)) - g_i^{\{j\}}(C_h) \right\|_2^2, \tag{3}$$

where $\varepsilon \sim \boldsymbol{N(0,1)}$ is the Gaussian noise, i and h are randomly sampled from 1 to p and 1 to k, respectively, C_h is one of the clusters of class r, and $l = \min(l_1, l_2, \ldots, l_p)$. The architecture of $\mathcal{G}_{\theta_1}$ with parameters θ_1 following [7], composed of convolutional blocks, batch normalization, and rectified linear units. We chose random architecture from the model pool because we expect the extracted features to be more diverse. The loss functions utilized for the generator and discriminator for a certain class r are:

$$\begin{aligned} L_{\mathcal{G}_{\theta_1}} = {} & \mathbb{E}_{\varepsilon\sim\boldsymbol{N(0,1)}}[\log(1 - \mathcal{D}_{\theta_2}(\mathcal{G}_{\theta_1}(\varepsilon, r)))] \\ & + \mathbb{E}_{\boldsymbol{x}\sim P(r)}\left[\lambda_1 \cdot d \cdot \left(\mathrm{SM}\left(g_i(\mathcal{G}_{\theta_1}(\varepsilon, r))\right)\right.\right. \\ & \left.\left. - \mathrm{SM}\left(g_i(\boldsymbol{x})\right)\right)^2\right], \end{aligned} \tag{4}$$

$$\begin{aligned} L_{\mathcal{D}_{\theta_2}} = {} & \mathbb{E}_{\boldsymbol{x}\sim P(r)}[\log(\mathcal{D}_{\theta_2}(\boldsymbol{x}))] \\ & + \mathbb{E}_{\boldsymbol{x'}\sim P_{\mathcal{G}_{\theta_1}}(\varepsilon, r)}[\log(1 - \mathcal{D}_{\theta_2}(\boldsymbol{x'}))], \end{aligned} \tag{5}$$

where $\mathcal{D}_{\theta_2}$ is the discriminator with parameters θ_2, SM is the softmax operator, $P(r)$ and $P_{\mathcal{G}_{\theta_1}}(\varepsilon, r)$ are the distribution of actual data and generated data of class r, respectively, λ_1 is the coefficient. The first term in (4) and (5) follow Condition GAN [41]. In our experiment, λ_1 is set as 2. The architecture of $\mathcal{D}_{\theta_2}$, following [7], is similar to $\mathcal{G}_{\theta_1}$, with the only difference being the substitution of batch normalization in $\mathcal{G}_{\theta_1}$ with layer normalization. We can draw from (4) that the larger the distance between the generated images and the cluster center, the larger the penalty to the generator, and thus $\mathcal{G}_{\theta_1}$ is constrained to generate more representative samples. The adaptive coefficient proposed by us is similar to the MiniMax$*$ function proposed in the following pragraph, both aiming to fine-tune the generative model to produce representative samples. The former is employed for low-resolution datasets, while the latter is used for high-resolution datasets. The use of GAN models to handle small datasets is attributed to the significantly smaller parameters of GAN compared to diffusion models and the superior quality of generated images compared to VAEs. The size of diffusion models can even exceed the storage space occupied by the small dataset itself.

2) Large-resolution datasets: Although Gu et al. [39] proposed a min-max loss to fine-tune the diffusion model, which addresses the representativeness and diversity of generated images to some extent, the optimization objective still leaves room for improvement. For example, considering the representativeness,

Gu et al. minimized the maximum distance between the features of the generative images and the real images. However, such strategy only consider the most extreme case, increasing the optimization difficulty. To this end, we first rank the cosine similarities between the features of generated images and those of real images, and then minimize the average of the largest K distances. The objective can be formally expressed as:

$$\min \frac{1}{K}\sum_{i=1}^{K}(1-d_{(i)}), \{d_{(i)}\}=\text{sort}\left(\{\cos(\boldsymbol{M_r}, \mathcal{G}_{\theta_1}(\boldsymbol{x_i}))\}\right)_{\text{asc}}, \tag{6}$$

where $\{d_{(i)}\} = \text{sort}(\{A\})_{\text{asc}}$ means that $d_i = A_i$ and $A_1 \leq A_2 \leq \ldots \leq A_B$, B is the batch size, and M_r is the stored real images in the memory.

Similarly, Gu et al. also maximized the minimum distance between the features of the generative images. We can still improve it by maximizing the average of the smallest K distances:

$$\max \frac{1}{K}\sum_{i=1}^{K} d_{(i)},\ \{d_{(i)}\} = \text{sort}\left(\{\cos(\boldsymbol{M_g}, \mathcal{G}_{\theta_1}(\boldsymbol{x_i}))\}\right)_{\text{dsc}}, \tag{7}$$

where $\{d_{(i)}\} = \text{sort}(\{A\})_{\text{dsc}}$ means that $d_i = A_i$ and $A_1 \geq A_2 \geq \ldots \geq A_B$, B is the batch size, M_g is the stored generated images in the memory, $K = 7$ in our experiment. The total loss function is L_{simple} for training the diffusion model and the losses defined in (6)–(7). In the experiments, such MiniMax-based extended version is denoted with an asterisk, such as MiniMax$*$ and Ours$*$.

D. Deployment Stage

Previous works adopting the GM to do DD always neglected careful design for the deployment phase. They generated an equal number of generated images as the target distilled images. The overall idea in this paper is that we first generate images more than the number of IPC, and then we conduct DD based on gradient matching loss on these redundant images. Although the overall design idea is the same no matter what the resolution of the dataset is, the specific designs for condensing low-resolution and high-resolution datasets are quite different due to different training strategies and different memory consumption. For the low-resolution datasets, we adopt DiM [7] as our baseline model and use GAN as the generator since GAN is usually smaller than the diffusion model. We adopt MiniMax [39] as our baseline model for the high-resolution ones since training GAN on a large dataset is quite unstable.

1) Low-resolution datasets: Following DiM, different images are generated for each evaluation epoch. The difference is that we ensemble these images across the epochs through gradient matching based DD. Since generating images by GAN is fast and consumes little memory, we adopt an online distillation strategy. In other words, we validate and integrate concurrently. Suppose the last preserved images are $\mathcal{T}_{cache}$ and images generated in the current epoch are $\mathcal{S}$. We consider $\mathcal{S} \cup \mathcal{T}_{cache}$ as the large dataset and employ a gradient matching strategy to update $\mathcal{S}$:

$$\mathcal{S} \leftarrow \mathcal{S} - \eta_{\mathcal{S}} \cdot \nabla_{\mathcal{S}} \| \nabla_\theta \mathcal{L}\left(f_{\theta^{\mathcal{S}\cup\mathcal{T}_{\text{cache}}}}(\bar{\boldsymbol{x}}), \bar{y}\right) - \nabla_\theta \mathcal{L}\left(f_{\theta^{\mathcal{S}}}(\boldsymbol{x'}), y'\right) \|_2, \tag{8}$$

Algorithm 1: Deploying for the Low-Resolution Dataset.

Input: A pre-trained GM $\mathcal{G}_{\theta_1}$, pre-trained neural network f on the original dataset, sliding window size $\bar{w}$, a variable to record current window size $CurW$, generated images for the current iteration $\mathcal{T}_0$ and saved images within the window $\mathcal{T}_{cache}$ with the operators $append(x)$ (add x to the last of $\mathcal{T}_{cache}$) and $pop(0)$ (pop the first element added to $\mathcal{T}_{cache}$).

Output: Synthetic dataset $\mathcal{S}$.

1: Set $CurW = 0$, $\mathcal{T}_0 = \varnothing$, $\mathcal{T}_{cache} = \varnothing$;
2: **for** $t_1 = 0$ to the number of evaluation epochs **do**
3: $\mathcal{T}_0 \leftarrow \mathcal{G}_{\theta_1}(\varepsilon, r)$ and $\mathcal{S} \sim \mathcal{T}_0$;
4: **if** $CurW = 0$ **then**
5: Set $CurW = 1$ and $\mathcal{T}_{cache} = \mathcal{T}_0$;
6: **else if** $CurW < \bar{w}$ **then**
7: $CurW \leftarrow CurW + 1$ and $\mathcal{T}_{cache}.append(\mathcal{T}_0)$;
8: **for** $t_2 = 0$ to the number of inner loops **do**
9: Update $\mathcal{S}$ by (8) for all the categories;
10: **end for**
11: **else**
12: $\mathcal{T}_{cache}.pop(0)$ and $\mathcal{T}_{cache}.append(\mathcal{T}_0)$;
13: **for** $t_2 = 0$ to the number of inner loops **do**
14: Update $\mathcal{S}$ by (8) for all the categories;
15: **end for**
16: **end if**
17: **end for**

where $\eta_{\mathcal{S}}$ is the learning rate for updating $\mathcal{S}$, $(\bar{\boldsymbol{x}}, \bar{y}) \in \mathcal{S} \cup \mathcal{T}_{cache}$, $(\boldsymbol{x'}, y') \in \mathcal{S}$ and $\mathcal{L}(f_{\theta^{\mathcal{S}}}(\boldsymbol{x}), y) = \frac{1}{\|\mathcal{S}\|}\sum_{x\in\mathcal{S}} \ell(f_{\theta^{\mathcal{S}}}(\boldsymbol{x}), y)$. We do not choose trajectory matching (TM) because TM requires storing a significant number of network training trajectories, leading to substantial storage space usage and prolonged preprocessing time. Although (8) can ensemble the features of these images, as the number of iterations increases, the stored images accumulate, requiring more and more GPU memory. Therefore, there is a risk of running out of memory. To solve the problem, we devise a sliding window based technique. Suppose the pre-set window size is $\bar{w}$ and the number of images stored in $\mathcal{T}_{cache}$ is larger than $\bar{w}$. We will drop the element that is added first to the $\mathcal{T}_{cache}$. Such data structure is similar to a queue. The complete algorithm is outlined in Algorithm 1. The first line of code aims to record the current window size and initialize the distilled and cached datasets for future use. We generate IPC$\times c$ images in line 3. In lines 4–16, we maintain the length variable $CurW$ for the window and perform distillation only within this window. We choose $\bar{w} = 10$ in our experiments. Since the strategy integrates the features of generated image sequences through distillation, it is referred to as the Sequential Ensembling by Distillation Strategy (SEDS).

To boost the performance further, we integrate the model via the exponential moving average (EMA) strategy: $h^i = \alpha f^{i-1} + (1-\alpha) f^i$ where α is 0.99 in our experiments and i is the training epoch. This is because new distilled images are generated in each round, and using the EMA strategy ensures that the classification model updates its parameters with the current

images while retaining the supervisory signals from previously distilled images.

2) High-resolution datasets: We adjusted two fronts to consider GPU memory size. Firstly, we adopted an offline distillation strategy, meaning we generated all the images and then compressed them. This approach decouples the generation and compression processes, reducing the demand for GPU memory. Secondly, instead of compressing in image space, we performed DD in a smaller-dimensional feature space. The compression ratio, calculated as $(3 \times 256 \times 256)/(4 \times 32 \times 32) = 48$, is remarkably significant since the shapes of images and features are $3 \times 256 \times 256$ and $4 \times 32 \times 32$, respectively. Specifically, we first use DDIM [16] to sample latent vectors whose transition kernel $q(\boldsymbol{x_{(s)}} \mid \boldsymbol{x_{(t)}}, \boldsymbol{x_{(0)}})$ between state t to s $(1 \leq s < t \leq M, t, s \in \mathbb{Z}^+)$ is:

$$q\left(\boldsymbol{x_{(s)}} \mid \boldsymbol{x_{(t)}}, \boldsymbol{x_{(0)}}\right) = \frac{\sqrt{\bar{\alpha}_s}}{\sqrt{\bar{\alpha}_t}}\left(\boldsymbol{x_{(t)}} - \sqrt{1-\bar{\alpha}_t}\varepsilon_{\theta_3}\left(\boldsymbol{x_{(t)}}, t\right)\right) + \sqrt{1-\bar{\alpha}_s}\varepsilon_{\theta_3}\left(\boldsymbol{x_{(t)}}, t\right), \bar{\alpha}_t = \prod_{t=1}^{M}(1-\beta_t), \tag{9}$$

where $\beta_1, \beta_2, \ldots, \beta_M$ are the pre-set variances schedule. $\varepsilon_{\theta_3}(\boldsymbol{x_{(t)}}, t)$ is the pre-trained denoising network with parameters θ_3 whose input is the t^{th} sampled image $\boldsymbol{x_{(t)}}$ and the sampling step t. M is the total sampling step. In our experiment, we set $M = 50$, $\beta_1 = 0.002$, $\beta_M = 0.4$ and $\beta_1, \beta_2, \ldots, \beta_M$ increase linearly. Suppose the generation results are $\mathcal{T}_0 = \bigcup_{i=1}^{n_0}(\boldsymbol{x_i}, y_i)$ where n_0 is often larger than the final required amount. After we randomly sampled from $\mathcal{T}_0$ as the initial distilled dataset $\mathcal{S}_0$, then we can update $\mathcal{S}_0$ through the following matching function:

$$L_{match} = \lambda_2 \cdot \left\|\nabla_\theta \mathcal{L}\left(f_{\theta^{\mathcal{T}_0}}(\boldsymbol{x}), y\right) - \nabla_\theta \mathcal{L}\left(f_{\theta^{\mathcal{S}_0}}(\boldsymbol{x'}), y'\right)\right\|_2, \tag{10}$$

$$\theta \leftarrow \theta - \eta_\theta \cdot \nabla_\theta \ell\left(f_{\theta^{\mathcal{T}_0}}(\boldsymbol{x}), y\right), \tag{11}$$

where f is a pre-trained classification network for a few epochs (we set the number of the pre-trained epochs as 3) rather than a well-trained one (pre-trained 300 epochs), and λ_2 is the coefficient. Note that the input for f is not an image but rather a latent vector. We set λ_2 as 1in our experiments. Following [29], the reason for doing this is that a network pre-trained for a few rounds can produce more diverse gradient information during gradient matching. Finally, we map the distilled latent vectors to the image space via a pre-trained VAE decoder. Now, the key question to this procedure is how to decide the initial number of generated images, that is, n_0. From our experiments, we found that a larger n_0 does not always result in superior outcomes. Additionally, as n_0 increases, the preprocessing time also increases. Therefore, in this algorithm, we use the top-1 accuracy achieved on the final distilled dataset as the criterion for choosing n_0. We start by trying a smaller n_0, and if there is an improvement in the final accuracy, we consider increasing the value of n_0. Through multiple experiments, we empirically determine that when IPC is set to 10/20/50, the corresponding values of n_0 are 750/1,000/1,000, respectively. A detailed analysis is presented in Section IV-D. The complete algorithm is depicted in Algorithm 2. The first line of code aims to record the accuracy of the model trained on $\mathcal{S}_0$, save initial generated latent vectors, sample $n_0 \times c$ Gaussian noise ε, generate $n_0 \times c$ latent vectors via DDIM and initialize $\mathcal{S}_0$ sampling from $\mathcal{T}_0$. In lines 3–7, we conduct gradient matching in the latent space where we iterate through all categories to update $\mathcal{S}_0$ and sample from the original latent vectors $\mathcal{T}_0$ to update the parameters of the classification model in lines 5 and 6, respectively. In lines 8–16, we devise a strategy that allows for dynamically adjusting the dataset size. Finally, we project the latent space to the image space.

Algorithm 2: Deploying for the High-Resolution Dataset.

Input: A pre-trained diffusion model Dif, a pre-trained VAE decoder D, pre-defined neural network f, initial number of generated latent vectors n_0, the number of classes c, the learning rate for updating both the synthetic dataset and the neural network $\eta_{\mathcal{S}}, \eta_\theta$, the evaluation interval ein and the increasing number of latent vector n_i.

Output: Synthetic dataset $\mathcal{S}$.

1: $Acc = 0$, $\mathcal{T}_0 \leftarrow Dif(\varepsilon, r)$, $\mathcal{S}_0 \sim \mathcal{T}_0$, $|\mathcal{T}_0| = n_0$;
2: Train f via latent-label pairs $(\mathcal{T}_0, r)$ for 3 epochs;
3: **for** $t_1 = 0$ to the number of outer loops **do**
4: **for** $t_2 = 0$ to the number of inner loops **do**
5: Update $\mathcal{S}_0$: $\mathcal{S}_0 \leftarrow \mathcal{S}_0 - \eta_{\mathcal{S}_0} \nabla_{\mathcal{S}_0} L_{match}$;
6: Update θ_{t_2} via (11);
7: **end for**
8: **if** t_1 can be divided by ein **then**
9: Acc_{t_1} is the accuracy of the f trained on $D(\mathcal{S}_0)$;
10: **if** Acc_{t_1} is larger than Acc **then**
11: $\mathcal{T}_0 = \mathcal{T}_b \leftarrow Dif(\varepsilon, r)$, $|\mathcal{T}_0| = n_0 + n_i$;
12: $Acc \leftarrow Acc_{t_1}$, $n_0 \leftarrow n_0 + n_i$;
13: **else**
14: Restore $\mathcal{T}_0$ as $\mathcal{T}_b$;
15: **end if**
16: **end if**
17: **end for**
18: $\mathcal{S} = D(\mathcal{S}_0)$; ▷ map the latent space to the image space.

E. Theoretical Analysis

Now, we will derive the generalization bound for our proposed DD algorithm based on McDiarmid's inequality [9]. Compared to the purely combinatorial notions like growth function and VC dimension [9], McDiarmid's inequality explicitly expresses the loss function and proves more convenient for our subsequent derivations.

Theorem 1: Suppose $\mathcal{F}$ is the set of loss function: $\mathcal{F} = \{L : \mathbb{R}^{3\times w\times h} \times \mathbb{R}^c \to \mathbb{R}\}$, the upper bound of $\forall L \in \mathcal{F}$ is at most a, $\tilde{\boldsymbol{X}} = \{\boldsymbol{\tilde{x_1}}, \boldsymbol{\tilde{x_2}}, \ldots, \boldsymbol{\tilde{x_{m'}}}\}$ is i.i.d sample-label pairs from dataset $\mathcal{S}$ where $\boldsymbol{\tilde{x_i}} = (\boldsymbol{x'_i}, y'_i) \in \mathcal{S}$, $\boldsymbol{\sigma} = (\sigma_1, \sigma_2, \ldots, \sigma_{m'})^\top$ where $\sigma_i \in \{-1, 1\}$ are independent uniform random variables, then $\forall \delta \in (0, 1)$ and $\forall \ell \in \mathcal{F}$ with possibility at least $(1 - \delta)$, we

TABLE II
EFFECT OF THE ORIGINAL DATASET AS A PROXY DATASET. "O, G, AND O+G" REPRESENT THE ORIGINAL DATASET, GENERATED IMAGES ALONE AND THE ORIGINAL DATASET COMBINED WITH GENERATED IMAGES, RESPECTIVELY. ALL THE EXPERIMENTS ARE CONDUCTED ON THE IMAGEWOOF DATASET.

Proxy dataset	IPC	Ratio	ConvNet6			ResNetAP10			ResNet18		
			O	G	O+G	O	G	O+G	O	G	O+G
25%	10	0.8%	30.3±1.3	37.2±0.4	**38.8±0.9**	33.7±0.5	40.6±0.6	**43.4±0.5**	34.1±0.3	41.1±0.3	**43.5±2.1**
	20	1.6%	36.7±0.5	40.2±1.0	**41.7±0.3**	39.9±0.5	**46.2±0.3**	44.6±1.1	38.3±0.7	43.1±0.4	**44.5±1.5**
	50	3.9%	46.6±0.6	**55.0±0.7**	52.2±0.7	51.7±0.6	**57.3±0.7**	57.1±0.5	50.2±0.7	**57.6±1.0**	55.7±0.8
50%	10	0.8%	32.4±0.3	**37.2±0.4**	34.8±0.9	34.0±1.5	**40.6±0.6**	37.9±0.5	35.6±0.2	**41.1±0.3**	39.8±0.7
	20	1.6%	37.1±1.2	40.2±1.0	**41.1±0.5**	40.1±0.4	**46.2±0.3**	43.5±0.6	37.9±0.7	43.1±0.4	**44.1±0.5**
	50	3.9%	48.7±1.1	**55.0±0.7**	51.7±0.8	52.0±0.6	**57.3±0.7**	56.3±0.4	50.1±0.5	**57.6±1.0**	56.9±1.1
75%	10	0.8%	30.7±0.7	**37.2±0.4**	35.2±0.9	33.3±0.8	**40.6±0.6**	38.5±0.8	34.3±0.4	**41.1±0.3**	40.3±0.2
	20	1.6%	34.2±1.1	40.2±1.0	**41.1±1.5**	40.1±0.7	**46.2±0.3**	44.0±0.2	37.4±1.0	43.1±0.4	**44.6±0.6**
	50	3.9%	46.7±1.4	**55.0±0.7**	53.9±0.3	53.9±1.1	**57.3±0.7**	56.4±1.3	51.1±1.5	**57.6±1.0**	55.5±0.5

have:

$$\mathbb{E}[\ell(\tilde{\boldsymbol{x}})] \leqslant \frac{1}{m'}\sum_{i=1}^{m'}\ell\left(\tilde{\boldsymbol{x}_i}\right)+2\mathbb{E}_{\boldsymbol{\sigma}}\left[\sup_{\ell\in\mathcal{F}}\frac{\langle\boldsymbol{\sigma}\cdot\ell(\tilde{\boldsymbol{X}})\rangle}{m'}\right]+3a\sqrt{\frac{\log\frac{2}{\delta}}{2m'}}, \tag{12}$$

where $\langle\cdot,\cdot\rangle$ is the inner product.

Detailed proof of Theorem 1 is shown in the supplementary materials. According to the Cauchy inequality and $\sqrt{a+b} \leqslant \sqrt{a}+\sqrt{b}, \forall a \geqslant 0, \forall b \geqslant 0$, we have:

$$\frac{\langle\boldsymbol{\sigma}\cdot\ell(\tilde{\boldsymbol{X}})\rangle}{m'} \leqslant \sqrt{\frac{\sum_{i=1}^{m'}\ell\left(\tilde{\boldsymbol{x}_i}\right)^2}{m'}} \leqslant \sqrt{\mathrm{Var}[\ell(\tilde{\boldsymbol{X}})]}+\mathbb{E}[\ell(\tilde{\boldsymbol{X}})]. \tag{13}$$

Then, according to (13) and the fact that the supremum of the sum is less than or equal to the sum of supremum, (12) can be transformed as:

$$\begin{aligned}\mathbb{E}[\ell(\tilde{\boldsymbol{x}})] \leqslant & \frac{1}{m'}\sum_{i=1}^{m'}\ell\left(\tilde{\boldsymbol{x}_i}\right)+2\sup_{\ell\in\mathcal{F}}\sqrt{Var\left[\ell(\tilde{\boldsymbol{X}})\right]} \\ & +2\sup_{\ell\in\mathcal{F}}\frac{\sum_{i=1}^{m'}\ell\left(\tilde{\boldsymbol{x}_i}\right)}{m'}+3a\sqrt{\frac{\log\frac{2}{\delta}}{2m'}}.\end{aligned} \tag{14}$$

Therefore, when the quantity of distilled images is the same for each class, thanks to our proposed algorithm in the deployment stage, the first and the third terms in the right hand of (14) are near the expected loss calculated on the original dataset according to the definition of DD:

$$\left\|\bar{\ell}\left(\mathcal{G}_{\theta_1}(\varepsilon)', y'\right)-\bar{\ell}(\boldsymbol{x}, y)\right\|_2 \leqslant \left\|\bar{\ell}\left(\mathcal{G}_{\theta_1}(\varepsilon), y\right)-\bar{\ell}(\boldsymbol{x}, y)\right\|_2, \tag{15}$$

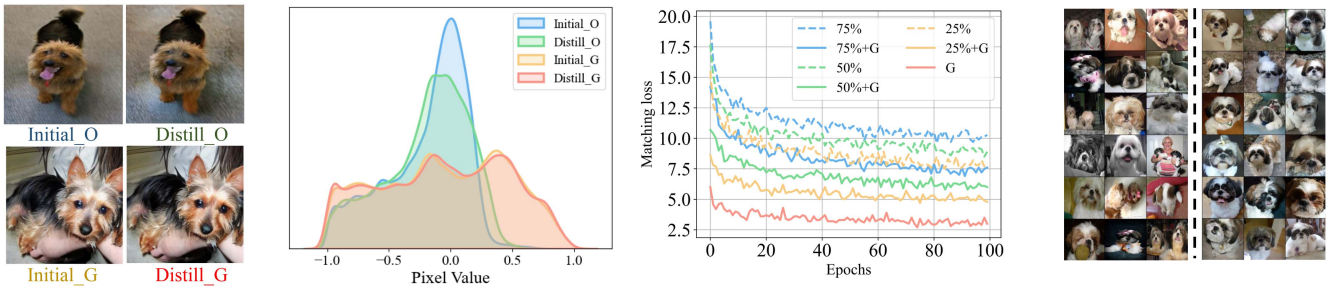
where $\mathcal{G}_{\theta_1}(\varepsilon)'$ is the condensed generated samples and $\bar{\ell}(\boldsymbol{x}, y) = \mathbb{E}[\ell(\boldsymbol{x}, y)]$ for brief. Therefore, the first and the third terms can be reduced. In addition, due to the adaptive coefficient in our proposed distillation stage or, similarly, the MiniMax$*$ loss function proposed in Sect. III-C(2), the second term in the right hand of (14) will also be reduced since the generator tends to generate samples concentrated at the center of each category. In conclusion, the upper bound of the generalization error $\mathbb{E}[\ell(\tilde{\boldsymbol{x}})]$ of our method is reduced thanks to our proposed components.

F. Discussion of the Proxy Dataset

Now, we explore the differences between distilling generated images and distilling original images, a point that has been overlooked in previous work. Specifically, we employed Dyn-Unc [42] on the ImageWoof dataset to select 25%, 50%, and 75% of the samples as a proxy dataset. These 3 different sets of samples are encoded using a VAE encoder to obtain latent vectors. Next, we explored 2 different scenarios: the first scenario involved directly distilling the latent vectors of the selected original dataset; the second scenario involved distilling by combining the latent vectors of the selected original dataset with samples generated by the diffusion model fintuned with MiniMax [39].

Table II studies the results of using the original images as a proxy dataset for secondary distillation. Regardless of the architectures used for evaluation or the number of distilled images, the performance of using raw samples as a proxy dataset is inferior to that of using a proxy dataset that includes generated images. For instance, on the ImageWoof dataset, using 50 distilled images per class with a 6-layer ConvNet, generated images outperform 25%, 50%, and 75% of the original images by 8.4%, 6.3%, and 8.3% in terms of distillation performance. When mixing generated images with original images as the proxy dataset, we found that a mix of 25% original images with generated images outperforms both 50% and 75% original images. This indicates that the generated images play a crucial role in the secondary distillation process.

Fig. 2(a) and (b) visualize the distillation results. In particular, Fig. 2(b) shows that the distillation results obtained from generated images are closer to the initialization images. Kernel density estimation, following [43], is employed to estimate the probability density function of image pixel values. Fig. 2(c) explains that phenomenon: the matching loss resulting from distillation using generated data is minimal. Moreover, regardless of the proportion of the pruned original dataset relative to the total dataset, the addition of generated samples consistently reduces the matching loss. Smaller matching losses lead to more stable image updates, resulting in better cross-architecture performance. Additionally, as the proportion of generated images in the proxy dataset increases, the matching loss decreases. We believe the cause of Fig. 2(c) lies in the fact that, for the same class, the similarity between generated images is higher than that



(a) Distilled Images (b) Kernel density estimation (c) Matching loss (d) Original proxy datasets

Fig. 2. Comparison of distilling the original dataset and the generated one. The four sub-images in (a) correspond precisely to the four curves in (b). (b) presents the kernel density estimation visualization results for the normalized images. "Initial_O" (for the original dataset) and "Initial_G" (for the generated dataset) represent the initialization, while "Distill_O" and "Distill_G" refer to the corresponding distilled results when distilling on the original or generated dataset, respectively. All the experiments are conducted on the ImageWoof dataset with IPC=10. 50% original images are selected for the proxy dataset for (a) and (b). "x%" represents the percentage of the original dataset and "G" means that the generated images were included in the training process in (c). The left side of the dashed line in (d) represents the original images, while the right side represents the generated images.

between original images, as illustrated in Fig. 2(d). The reason is that images generated by generative models fine-tuned with representativeness constraints exhibit lower background variation and more prominently emphasize class-specific features. More visualization can be found in the supplementary materials.

IV. Experimental Results

A. Dataset and Evaluation Metrics

We adopt SVHN [47], CIFAR10, CIFAR100 [48], TinyImageNet [49], ImageWoof, ImageNette [50], ImageBird, ImageCat [4], ImageNet10, ImageNet100 [51], and ImageNet1K in our experiments.

Similar to approaches in some previous work, the size of the distilled dataset is 1/10/50 in small datasets (SVHN, CIFAR10, CIFAR100, and TinyImageNet) and 10/20/50 in the subsets of ImageNet1K. Following acquiring the distilled dataset, we carried out three rounds of model training and reported the top-1 accuracy's mean and standard deviation. Moreover, to assess the robustness of the distilled data, we also perform experiments on cross-architecture generalization. This involves employing various network architectures in the distillation and validation processes.

B. Implementation Details

We adopt Pytorch [52] framework to implement our method. All the experiments were performed on a single NVIDIA GeForce RTX 4090 GPU. The implementation of various GANs is followed by StudioGAN [10]. The secondary distillation architecture for the diffusion-based DD is a four-layer convolution neural network (CNN). The evaluation architectures are six-layer CNN, ten-layer ResNet with the strided convolution replaced by average pooling, and eighteen-layer ResNet. We adopt two baselines in our experiments, *i.e.*, DiM [7] for SVHN, CIFAR10, CIFAR100, and TinyImageNet, and MiniMax diffusion [39] for the ImageNet1K and its subsets. During evaluation, when IPC is set to 1/10/50, the batch size is configured as 10/20/50. The inner loops of secondary distillation for GAN and diffusion are 1 and 10, respectively. The outer loops of secondary distillation for GAN and diffusion are 2 and 400, respectively. We use SGD with a learning rate of 0.01 and momentum of 0.5 for small datasets. For large datasets, the optimizer remains SGD with a learning rate of 0.05 and momentum of 0.5 during second-stage distillation.

C. Comparisons With State-of-The-Art Methods

1) SVHN and CIFAR10 Datasets: The quantitative comparison between our method and other methods on the SVHN and CIFAR10 datasets is shown in Table III. To achieve optimal results, the original DiM fine-tuned hyperparameters on each dataset (e.g., learning rate, batch size, etc.). In contrast, the method we reproduced did not undergo meticulous hyperparameter tuning and utilized all default parameters. As a result, our reproduced method falls short of the results presented in the paper. However, even with this limitation, when incorporating the proposed techniques, we still significantly outperformed the original paper's results, especially on the CIFAR10 dataset. Specifically, on the CIFAR10 dataset, our algorithm outperforms the original algorithm by 0.5%, 3.6%, and 1.5% when IPC is set to 1/10/50, respectively. Additionally, the standard deviation of our algorithm is smaller than that of the original algorithm, demonstrating the effectiveness and stability of our approach. In addition, leveraging the same baseline, DiM, our method outperforms Li et al. [38] by 0.2%, 1.6%, and 0.1% on the CIFAR10 dataset for IPC values of 1, 10, and 50, respectively.

The cross-architecture experiments on the SVHN and CIFAR10 datasets is shown in Table IV. Two unseen architectures (ResNet10 and ResNet18) were used during the evaluation stage. Our method shows good cross-architecture performance. The distillation performance evaluated on the ConvNet3 architecture between our method and factorization-based DD is shown in Table V. Only single-channel images are saved in HaBa [46], and a single image contains four sub-images in IDC [5]. Therefore, our method's forward number of images is increased for a fair comparison. Our method achieves higher performance than HaBa and IDC, and thus shows effectiveness.

2) CIFAR100 Dataset: The CIFAR100 dataset poses a challenge for GM-based DD algorithms because the number of

TABLE III
COMPARISON OF ACCURACY OF OUR METHOD WITH OTHER STATE-OF-THE-ART CORESET SELECTION METHODS (K-C [19], HERD [20]) AND DD METHODS (DC [3], DC+B [35], DSA [44], CAFE [45], DATADAM [26], IDM [27], IID [28], KIP [22], MTT [4], DIM [7]) ON THE SVHN, CIFAR10, CIFAR100, AND TINY IMAGENET DATASETS. "K-C" INDICATES K-CENTER AND "HERD" INDICATES HERDING. ‡ INDICATES CAFE WITH THE DSA STRATEGY. THE BASELINE MODEL OF IID IS IDM. † INDICATES THAT THE MEAN AND STANDARD DEVIATION OF THE ACCURACY ARE REPRODUCED BY US ACCORDING TO THE PUBLIC CODE. THE ACCURACY TRAINING ON THE FULL SVHN, CIFAR10, CIFAR100, AND TINY IMAGENET DATASETS IS 95.4±0.1, 84.8±0.1, 56.2±0.3 AND 37.6±0.4, RESPECTIVELY.

Dataset	IPC	Method														
		Random	K-C	Herd	DC	DC+B	DSA	CAFE	CAFE‡	DataDAM	IDM	IID	KIP	MTT	DiM	DiM †	Ours
SVHN	1	14.6±1.6	21.0±1.5	20.9±1.3	31.2±1.4	32.2±0.3	27.5±1.4	42.6±3.3	42.9±3.0	-	-	66.3±0.1	57.3±0.1	58.5±1.4	80.9±1.2	76.6±1.8	**81.6±0.4**
	10	35.1±4.1	14.0±1.3	50.5±3.3	76.1±0.6	76.4±0.5	79.2±0.5	75.9±0.6	77.9±0.6	-	-	82.1±0.3	75.0±0.1	70.8±1.8	**87.9±0.5**	83.9±1.1	86.9±0.3
	50	70.9±0.9	20.1±1.4	72.6±0.8	82.3±0.3	82.8±0.6	84.4±0.4	81.3±0.3	82.3±0.4	-	-	85.1±0.5	80.5±0.1	85.7±0.1	**90.4±0.3**	85.8±0.8	**88.2±0.3**
CIFAR10	1	14.4±2.0	21.5±1.3	21.5±1.3	28.3±0.5	30.5±0.3	28.8±0.7	30.3±1.1	31.6±0.8	32.0±1.2	45.6±0.7	47.1±0.1	49.9±0.2	46.3±0.8	51.3±1.0	47.8±1.2	**51.8±0.6**
	10	26.0±1.2	14.7±0.9	31.6±0.7	44.9±0.5	45.2±0.4	52.1±0.5	46.3±0.6	50.9±0.5	54.2±0.8	58.6±0.1	59.9±0.2	62.7±0.3	65.3±0.7	66.2±0.5	65.3±0.9	**69.8±0.3**
	50	43.4±1.0	27.0±1.4	40.4±0.6	53.9±0.5	54.9±0.3	60.6±0.5	55.5±0.6	62.3±0.4	67.0±0.4	67.5±0.1	69.0±0.3	68.6±0.2	71.6±0.2	72.6±0.4	71.1±0.5	**74.1±0.3**
CIFAR100	1	4.2±0.3	8.3±0.3	8.4±0.3	12.8±0.3	13.7±0.7	13.9±0.3	12.9±0.3	14.0±0.3	14.5±0.5	20.1±0.3	24.6±0.1	15.7±0.2	24.3±0.3	-	26.6±0.5	**29.4±0.5**
	10	14.6±0.5	7.1±0.2	17.3±0.3	25.2±0.3	26.0±0.5	32.3±0.3	27.8±0.3	31.5±0.2	34.8±0.5	45.1±0.1	**45.7±0.4**	28.3±0.1	40.1±0.4	-	37.6±0.1	40.5±0.5
	50	30.0±0.4	30.5±0.2	33.7±0.5	-	-	42.8±0.4	37.9±0.3	42.9±0.2	49.4±0.3	50.0±0.2	**51.3±0.4**	-	47.7±0.2	-	40.3±0.4	44.6±0.2
TinyImageNet	1	1.4±0.1	1.6±0.1	2.8±0.2	5.3±0.1	11.4±0.2	5.7±0.1	-	-	8.3±0.4	10.1±0.2	10.0±0.1	-	6.2±0.4	-	14.9±0.1	**16.6±0.2**
	10	5.0±0.2	5.1±0.2	6.3±0.2	12.9±0.1	22.9±0.4	16.3±0.2	-	-	18.7±0.3	21.9±0.2	23.3±0.1	-	17.3±0.2	-	22.7±0.1	**24.3±0.2**
	50	15.0±0.4	15.0±0.3	16.7±0.3	12.7±0.4	-	5.3±0.2	-	-	**28.7±0.3**	27.7±0.3	27.5±0.3	-	26.5±0.3	-	24.2±0.2	26.1±0.1

TABLE IV
CROSS ARCHITECTURE PERFORMANCE ON THE SVHN AND CIFAR10 DATASETS

Dataset	IPC	ResNet10			ResNet18		
		DiM	Ours	Full	DiM	Ours	Full
SVHN	1	48.9±5.5	**60.4±4.4**		63.6±2.9	**72.2±1.1**	
	10	82.4±0.2	**83.1±0.1**	96.0±0.1	**83.6±0.4**	83.4±0.2	96.4±0.4
	50	**86.6±0.2**	86.4±0.3		86.4±0.2	**86.5±0.1**	
CIFAR10	1	40.8±1.2	**42.3±0.8**		39.6±0.8	**41.9±0.5**	
	10	66.1±1.0	**68.0±0.3**	92.6±0.1	67.4±1.6	**68.3±0.3**	94.3±0.1
	50	72.5±0.4	**74.7±0.2**		73.8±0.3	**74.9±0.2**	

TABLE V
DISTILLATION PERFORMANCE AMONG THOSE FACTORIZATION-BASED DD METHODS. † INDICATES THAT THE MEAN AND STANDARD DEVIATION OF THE ACCURACY IS DRAWN FROM [7].

Dataset	IPC	Method				Full Dataset
		HaBa [46]	IDC [5]	DiM [7]	Ours	
SVHN	1	69.8±1.3	68.5†	80.0±0.7	**83.8±0.4**	95.4±0.1
	10	83.2±0.4	**87.5**†	84.4±1.1	87.1±0.4	
	50	88.3±0.1	**90.1**†	86.4±0.7	87.3±0.2	
CIFAR10	1	48.3±0.8	50.6†	53.6±0.8	**57.3±0.5**	84.8±0.1
	10	69.9±0.4	67.5±0.5	68.3±0.8	**71.1±0.3**	
	50	74.0±0.2	74.5±0.1	73.0±0.2	**74.5±0.1**	

images per class is relatively small compared to other datasets; for example, it contains only one-tenth the number of images per class as CIFAR10. Even so, the proposed method still achieves an accuracy of 29.4% when IPC=1, which represents a state-of-the-art result. However, as IPC increases, the amount of class-specific information that a generative model can provide quickly becomes saturated, and generating additional images no longer brings further benefits.

3) TinyImageNet Dataset: Similar to CIFAR100, we choose BigGAN as our baseline model since the classification accuracy is only 15.3%±0.2% when IPC=50 via SAGAN. As shown in Table III, when IPC is 1 and 10, our method surpasses IDM by 6.5% and 2.4%, respectively, surpasses IID by 6.6% and 1.0%, respectively and surpasses our baseline model DiM by 1.7% and 1.6%, respectively. Therefore, the effectiveness of our method is once again confirmed on small datasets.

4) ImageWoof Dataset: Table VI summerizes the cross-architecture performance results of our method and other state-of-the-art methods on the ImageWoof dataset. While our method utilizes the ConvNet4 network in the distillation phase, Table VI indicates that our approach is robust to changes in architecture. Furthermore, the secondary distillation based on the diffusion model fine-tuned with the proposed extended MiniMax loss, denoted as Ours∗, shows clear advantages over the original MiniMax-based secondary distillation when the IPC is large. For instance, when IPC=20 and ResNet18 is used for evaluation, Ours∗ outperforms the original MiniMax by 4.6%.

In addition to the ResNet for cross-architecture experiments, more diverse architectures, such as VGG [53], ViT [54], and EfficientNet [55] are tested in Table VII. Detailed architectures are VGG11, ViT-B-16, and EfficientNet-B0 sourced from the torchvision library. From the results on cross-architecture generalization, we can observe that Ours∗ achieves the best performance in most cases particularly when IPC is large, evaluating with VGG11 and EfficientNet-B0 architectures. On ViT, it attains performance comparable to that of Ours. For RDED, when the IPC value is relatively low, such as 1, 2, 5, or 10, our method delivers the best results. However, for higher IPC values like 20 or 50, our method performs worse than RDED on the VGG and EfficientNet architectures, but still surpasses RDED by 4.5% on the ViT architecture. This discrepancy is due to the fact that, even with full dataset training, the accuracy of the ViT architecture only reaches 60.3%, which is significantly lower than the 89.7% of VGG and 89.3% of EfficientNet. RDED, on the other hand, heavily relies on pre-training the model on the full dataset, where the pre-trained model acts as a teacher to train the student model using Kullback-Leibler Divergence. As the pre-trained model performs well across the entire dataset, the advantages of RDED become more pronounced with increasing IPC. However, when the pre-trained model performs poorly, the performance of RDED deteriorates accordingly. In contrast, our

TABLE VI
COMPARISON OF ACCURACY OF OUR METHOD WITH SELECTION-BASED METHODS (K-CENTER [19] AND HERDING [20]), DD METHODS (SRe ^{2} L [31], RDED [33], DM [25], IDC [5], GLaD [6] AND MINIMAX [39]) AND A GENERATIVE MODEL DiT [17] ON THE IMAGEWOOF DATASET. † INDICATES THE MEAN AND STANDARD DEVIATION OF THE ACCURACY ARE REPRODUCED BY US ACCORDING TO THE PUBLICLY DISTILLED DATASET. THE OTHER RESULTS ARE DRAWN FROM MINIMAX [39]. ‡ MEANS SOFT LABELS ARE NOT ADOPTED. "X-1" MEANS THE NUMBER OF SUBGRAPHS OF ONE DISTILLED IMAGE IS ONE, SUCH AS "RDED-1" AND "IDC-1". "OURS (D)" AND "OURS" INDICATE THE PERFORMANCE OF SECONDARY DISTILLATION WITH THE DISTRIBUTION MATCHING AND GRADIENT MATCHING STRATEGY, RESPECTIVELY. "MINIMAX∗" USES THE PROPOSED EXTENDED MINIMAX LOSS, AND "OURS∗" FURTHER COMBINES IT WITH SECONDARY DISTILLATION. THE ACCURACY TRAINING ON THE FULL DATASET VIA CONVNET6, RESNETAP10, AND RESNET18 IS 86.4±0.2, 87.5±0.5, AND 89.3±1.2.

IPC	Ratio	Test Model	Method											
			K-Center	Herding	SRe^{2}L‡	RDED-1‡	DiT	DM	IDC-1	GLaD	MiniMax	MiniMax*	Ours (D)	Ours	Ours*
10	0.8%	ConvNet6	19.4±0.9	26.7±0.5	18.1±0.5	24.1±1.2	34.2±1.1	26.9±1.2	33.3±1.1	33.8±0.9	37.0±1.0	34.3±0.6	34.9±0.9	**37.2±0.4**	**37.2±1.7**
		ResNetAP10	22.1±0.1	32.0±0.3	18.4±0.2	28.2±0.5	34.7±0.5	30.3±1.2	39.1±0.5	32.9±0.9	39.2±1.3	36.7±0.3	40.2±0.3	**40.6±0.6**	39.9±1.2
		ResNet18	21.1±0.4	30.2±1.2	15.1±0.2	29.1±0.6	34.7±0.4	33.4±0.7	37.3±0.2	31.7±0.8	37.6±0.9	35.6±1.7	39.6±1.0	**41.1±0.3**	**41.1±1.0**
20	1.6%	ConvNet6	21.5±0.8	29.5±0.3	18.0±0.3	31.4±0.6	36.1±0.8	29.9±1.0	35.5±0.8	-	37.6±0.2	39.2±0.7	39.1±1.6	40.2±1.0	**42.5±0.8**
		ResNetAP10	25.1±0.7	34.9±0.1	19.3±0.2	36.1±0.9	41.1±0.8	35.2±0.6	43.4±0.3	-	45.8±0.5	46.2±0.4	43.7±0.9	46.2±0.3	**46.6±0.7**
		ResNet18	23.6±0.3	32.2±0.6	16.9±0.2	33.7±1.0	40.5±0.5	29.8±1.7	38.6±0.2	-	42.5±0.6	42.9±0.9	43.0±1.3	43.1±0.4	**47.1±1.3**
50	3.8%	ConvNet6	36.5±1.0	40.3±0.7	20.0±0.4	40.1±1.7	46.5±0.8	44.4±1.0	43.9±1.2	-	53.9±0.6	52.9±0.5	54.9±0.6	55.0±0.7	**56.0±1.5**
		ResNetAP10	40.6±0.4	49.1±0.7	19.9±0.6	47.9±0.5	49.3±0.2	47.1±1.1	48.3±1.0	-	56.3±1.0	60.1±1.7	57.8±1.2	57.3±0.7	**59.3±1.0**
		ResNet18	39.6±1.0	48.3±1.2	18.1±0.7	46.0±0.9	50.1±0.5	46.2±0.6	48.3±0.8	-	57.1±0.6	55.6±0.4	57.5±0.8	57.6±1.0	**61.2±1.9**
70	5.4%	ConvNet6	38.6±0.7	46.2±0.6	21.4±0.6	44.5±0.7	50.1±1.2	47.5±0.8	48.9±0.7	-	55.7±0.9	54.0±0.9	56.7±1.2	**58.2±0.7**	57.5±0.5
		ResNetAP10	45.9±1.5	53.4±1.4	21.1±0.2	52.9±1.1	54.3±0.9	51.7±0.8	52.8±1.8	-	58.3±0.2	59.5±1.4	58.7±0.6	59.6±0.7	**60.7±0.3**
		ResNet18	44.6±1.1	49.7±0.8	19.1±1.1	50.3±0.4	51.5±1.0	51.9±0.8	51.1±1.7	-	58.8±0.7	58.7±0.1	59.2±0.9	60.0±0.8	**63.8±0.3**
100	7.7%	ConvNet6	45.1±0.5	54.4±1.1	22.5±0.4	50.2±0.0	53.4±0.3	55.0±1.3	53.2±0.9	-	60.1±0.5†	58.3±1.0	61.1±0.5	**61.3±0.6**	61.1±1.3
		ResNetAP10	54.8±0.2	61.7±0.9	21.0±0.7	56.9±1.3	58.3±0.8	56.4±0.8	56.1±0.9	-	63.5±0.1†	63.8±0.4	63.7±0.5	64.4±0.6	**65.1±0.8**
		ResNet18	50.4±0.4	59.3±0.7	20.3±0.6	56.9±1.2	58.9±1.3	60.2±1.0	58.3±1.2	-	65.0±1.3†	62.3±0.7	64.7±0.8	65.3±0.7	**68.1±1.1**

TABLE VII
COMPARISON ACCURACY OF OUR METHOD WITH OTHER STATE-OF-THE-ART DD METHODS EVALUATING ON MORE ARCHITECTURES ON THE IMAGEWOOF DATASET

IPC	Ratio	VGG11						ViT						EfficientNet-B0					
		DiT	MiniMax	RDED	Ours (D)	Ours	Ours*	DiT	MiniMax	RDED	Ours (D)	Ours	Ours*	DiT	MiniMax	RDED	Ours (D)	Ours	Ours*
1	0.1%	17.8±1.3	18.6±0.7	14.8±1.0	19.1±1.1	**19.4±0.9**	19.3±0.6	21.1±0.5	15.7±0.7	14.7±0.8	15.4±0.3	**20.2±0.7**	13.7±0.4	**21.7±0.8**	17.1±0.6	16.9±1.0	18.1±0.9	21.3±0.3	18.0±0.2
2	0.2%	18.2±1.1	19.3±1.1	16.7±0.9	19.9±1.7	19.8±0.7	**20.8±1.4**	21.1±0.8	21.5±0.2	15.9±0.6	20.4±0.7	**22.3±0.2**	21.2±0.3	20.5±0.7	19.7±0.5	18.3±0.6	20.0±0.7	20.7±0.4	**22.1±0.2**
5	0.4%	23.7±0.7	22.6±0.6	18.7±1.4	23.5±1.0	23.9±0.8	**23.9±0.1**	27.8±0.3	28.9±0.2	22.7±0.6	28.1±0.5	**29.9±0.5**	28.3±1.5	24.5±0.3	26.9±0.8	25.1±0.8	28.8±1.0	27.7±0.3	**32.0±0.7**
10	0.8%	28.1±1.7	26.7±1.4	24.1±0.6	31.4±1.1	28.2±1.6	**31.2±1.8**	34.3±0.8	34.7±0.2	27.0±1.2	35.5±0.8	**36.0±0.4**	33.9±0.5	30.9±1.6	31.6±0.9	35.1±0.2	36.5±1.1	35.1±1.0	**37.1±1.1**
20	1.6%	30.3±0.4	31.0±1.0	35.3±1.2	36.3±1.1	31.8±1.7	**36.2±1.3**	35.9±1.1	38.9±0.4	31.4±0.8	39.9±0.7	39.0±0.7	**40.2±0.3**	36.1±1.4	39.7±1.2	**52.3±0.5**	42.3±1.6	40.9±1.5	42.9±0.5
50	3.8%	43.7±0.7	48.2±0.3	**55.0±1.1**	51.0±2.6	50.5±0.5	50.5±0.5	42.7±0.5	44.5±0.6	40.8±1.2	**45.7±0.2**	45.3±0.5	44.9±1.7	45.0±0.9	45.8±1.0	**67.8±0.3**	54.3±1.1	51.1±0.9	53.7±1.4
Full	100%	89.7±0.4						60.3±0.7						89.3±0.6					

TABLE VIII
COMPARISON WITH THE RDED-1 [33] ALGORITHM ON THE IMAGEWOOF DATASET. "X-1" MEANS THE NUMBER OF SUBGRAPHS OF ONE DISTILLED IMAGE IS ONE.

IPC	Ratio	VGG11		ViT		EfficientNet-B0	
		RDED-1	Ours	RDED-1	Ours	RDED-1	Ours
1	0.1%	13.3±1.2	**19.4±0.9**	15.6±1.2	**20.2±0.7**	13.9±1.1	**21.3±0.3**
2	0.2%	16.5±0.5	**19.8±0.7**	17.7±1.0	**22.3±0.2**	19.0±1.2	**20.7±0.4**
5	0.4%	20.7±1.2	**23.9±0.8**	17.9±0.5	**29.9±0.5**	21.4±0.5	**27.7±0.3**
10	0.8%	22.8±2.8	**28.2±1.6**	18.7±1.1	**36.0±0.4**	26.4±1.8	**35.1±1.0**
20	1.6%	24.2±2.2	**31.8±1.7**	26.0±0.7	**39.0±0.7**	34.3±1.5	**40.9±1.5**
50	3.8%	33.1±0.7	**50.5±0.5**	30.3±1.2	**45.3±0.5**	45.4±0.4	**51.1±0.9**

method remains independent of the quality of the pre-trained model.

Since each distilled image in RDED [33] contains four sub-figures, the actual number of distilled images is larger than that in our method when the IPC is high. To ensure a fair comparison, we set the number of sub-figures in the RDED algorithm to one (denoted as RDED-1) and additionally conducted experiments on the ImageWoof dataset. The experimental results are shown in Table VIII. It can be observed that even under high-IPC settings, our method still significantly outperforms the RDED-1 algorithm. For example, using the VGG11 architecture, our method achieves a 17.4% higher accuracy than RDED-1 when IPC = 50. This further demonstrates the effectiveness of our proposed approach. Therefore, the reason why our method lags behind RDED at IPC = 50 is due to the difference in the number of distilled images, rather than the intrinsic performance of the method itself.

5) More Subsets of ImageNet1K Dataset: Table IX shows the distillation results on other subsets of ImageNet1K. Secondary distillation based on the original MiniMax loss (Ours) or our extended MiniMax loss (Ours∗) show clear advantages over the original MiniMax method. For example, on the ImageNette dataset when IPC=10, using ResNet18 as the evaluation architecture, the distillation performance of Ours∗ is 4.4% higher than that of MiniMax.

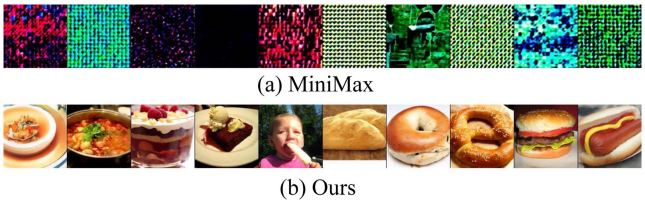
6) ImageNet1K Dataset: Due to the instability of MiniMax, fine-tuning the generative model on ImageNet1K leads to collapse and poor generation quality (Fig. 3). Using MiniMax∗ does not lead to collapse; however, its performance is still inferior to that of DiT. MiniMax requires splitting ImageNet1K into groups of 10 (or 20) classes, leading to 100 (or 50) training sessions and occupying 250 GB (or 125 GB) of disk space. To avoid this complexity, we used the DiT. A similar situation occurred with

TABLE IX
COMPARISON ACCURACY OF OUR METHOD WITH OTHER STATE-OF-THE-ART DD METHODS ON THE IMAGENETTE, IMAGEBIRD, IMAGECAT, IMAGENET10, AND IMAGENET100 DATASETS. "CONVNET6, RESNETAP10 AND RESNET18" ARE THE DEPLOYMENT ARCHITECTURES WHICH ARE THE SAME AS [39].

Dataset	IPC	Ratio	ConvNet6					ResNetAP10					ResNet18				
			DiT	MiniMax	MiniMax*	Ours	Ours*	DiT	MiniMax	MiniMax*	Ours	Ours*	DiT	MiniMax	MiniMax*	Ours	Ours*
ImageNette	10	0.8%	59.4±0.7	55.3±0.8	55.7±0.9	**59.7±0.9**	58.0±0.3	59.1±0.7†	62.0±0.2	61.8±1.1	62.1±0.2	**63.1±0.6**	61.9±1.3	58.7±0.6	58.5±1.2	60.7±1.1	**63.1±0.9**
	20	1.6%	63.2±0.4	62.8±0.9	62.4±0.4	63.1±0.6	**63.9±1.0**	64.8±1.2†	67.9±1.1	68.1±0.4	**68.9±0.7**	68.2±1.0	65.3±0.2	65.4±0.8	**68.1±1.5**	67.2±1.2	67.1±0.9
	50	3.9%	74.7±0.8	74.8±0.7	75.2±0.3	**77.5±1.5**	76.5±0.4	73.3±0.9	74.8±1.1	75.4±0.9	75.3±0.8	**76.1±0.3**	73.3±0.5	74.5±0.6	75.5±0.1	**76.8±0.3**	75.6±0.5
	Full	100%	95.5±0.1					95.7±0.1					96.5±0.4				
ImageBird	10	0.8%	49.7±0.8	48.5±1.4	46.7±1.0	52.1±0.3	**54.3±0.5**	53.6±0.7	54.9±1.5	53.1±0.8	55.3±0.4	**54.9±0.5**	54.3±1.7	53.8±1.3	52.7±1.0	56.5±0.1	**57.3±0.7**
	20	1.5%	54.8±1.3	57.5±0.8	57.1±1.1	**57.5±1.4**	56.7±0.3	59.9±0.7	62.0±0.5	60.9±0.4	**63.0±0.7**	61.8±0.2	60.4±0.5	58.7±1.2	61.1±0.8	61.2±1.1	**61.5±0.8**
	50	3.8%	68.1±0.7	71.3±1.0	69.9±0.8	71.9±0.4	**72.9±0.8**	70.4±0.9	72.3±0.7	70.7±0.4	72.9±0.3	**74.1±0.5**	69.9±0.7	72.7±1.0	72.0±0.4	73.6±1.6	**74.1±0.2**
	Full	100%	97.2±0.3					97.2±0.3					98.0±0.2				
ImageCat	10	0.8%	38.7±0.5	37.7±0.8	41.3±0.7	41.3±0.9	**43.9±0.5**	41.5±1.0	40.5±0.3	44.9±0.3	44.3±1.4	**45.7±0.6**	42.2±1.3	39.7±0.2	45.1±0.5	45.7±1.3	**45.7±0.6**
	20	1.5%	42.1±0.5	42.7±0.8	44.2±0.3	44.7±0.6	**45.6±0.2**	47.6±0.6	47.7±1.2	48.5±1.0	**49.7±0.6**	49.2±0.2	46.3±1.1	46.3±1.0	46.0±0.7	48.0±1.3	**48.9±0.3**
	50	3.8%	56.5±0.2	57.3±1.0	58.6±0.4	**58.7±0.5**	58.3±1.2	57.1±0.2	56.8±1.4	57.9±1.3	57.7±0.6	**58.6±0.7**	55.7±0.8	56.1±0.9	**58.1±0.2**	57.9±0.8	57.2±1.1
	Full	100%	77.7±1.0					78.1±0.4					79.1±0.7				
ImageNet10	10	0.8%	51.9±0.5	51.1±0.8	52.3±0.5	52.7±1.4	**53.9±1.4**	54.1±0.4†	53.1±0.2	54.9±1.0	55.9±0.6	**56.7±0.4**	50.4±1.2	48.7±1.4	52.9±0.8	53.5±0.8	**53.7±1.1**
	20	1.6%	56.4±0.9	54.7±0.1	54.7±0.2	**56.2±0.4**	54.6±0.2	58.9±0.2†	59.0±0.4	60.1±1.2	**60.9±1.1**	59.9±1.1	57.1±0.3	59.3±1.6	57.7±1.2	59.5±0.9	**59.6±1.3**
	50	3.9%	66.7±0.1	**73.0±1.8**	70.6±1.0	71.1±0.4	72.4±0.6	64.3±0.6	69.6±0.2	69.9±0.3	69.8±0.3	**70.9±1.4**	64.7±0.3	71.2±0.7	70.3±0.6	71.6±0.7	**72.1±0.2**
	Full	100%	93.7±0.2					94.9±0.4					95.1±0.4				
ImageNet100	10	0.8%	22.2±0.4	22.3±0.5	22.3±0.5	23.0±0.4	**23.0±0.4**	23.0±0.1	24.8±0.2	**24.8±0.2**	24.0±0.2	24.0±0.2	22.8±0.3	22.5±0.3	22.5±0.3	23.8±0.5	**23.8±0.5**
	Full	100%	79.9±0.4					80.3±0.2					81.8±0.7				

TABLE X
COMPARISON OF THE ACCURACY OF OUR METHOD WITH OTHER STATE-OF-THE-ART DD METHODS (SRE ^{2} L [31], GVBSM [32], RDED [33], AND DIT [17]) ON THE IMAGENET1K DATASET, EVALUATED ON VGG, VIT, AND EFFICIENTNET ARCHITECTURES.

IPC	Ratio	VGG11						ViT						EfficientNet-B0					
		SRe^{2}L	GVBSM	RDED	DiT	Ours	Ours (D)	SRe^{2}L	GVBSM	RDED	DiT	Ours	Ours (D)	SRe^{2}L	GVBSM	RDED	DiT	Ours	Ours (D)
1	0.1%	0.3±0.0	0.2±0.1	2.1±0.2	4.8±0.1	5.2±0.0	**5.4±0.0**	0.5±0.0	0.3±0.0	3.1±0.1	3.5±0.1	3.7±0.0	**3.7±0.1**	2.8±0.1	1.5±0.1	3.0±1.0	3.8±0.4	**4.6±0.5**	4.0±0.1
2	0.2%	0.6±0.0	0.4±0.1	3.7±0.0	9.7±0.1	10.3±0.2	**10.4±0.1**	0.6±0.1	0.5±0.0	5.1±0.1	6.2±0.0	6.5±0.1	**6.5±0.1**	6.8±0.1	3.9±0.1	7.9±1.3	8.2±0.3	**9.7±0.1**	8.8±0.3
5	0.4%	3.1±0.1	1.1±0.0	13.5±0.0	21.4±0.1	21.7±0.1	**21.7±0.0**	0.8±0.3	0.7±0.1	11.0±0.3	11.1±0.1	11.4±0.3	**11.4±0.1**	20.4±0.4	17.0±0.3	34.3±0.2	35.4±0.9	36.8±1.1	**36.8±0.6**
10	0.8%	8.3±0.3	2.6±0.1	20.3±0.3	30.1±0.2	**30.2±0.1**	30.0±0.1	2.8±0.5	1.3±0.2	19.2±0.4	20.6±0.5	**20.7±0.4**	20.5±0.6	29.5±0.1	33.8±0.1	44.6±0.3	45.6±0.8	45.9±0.8	**46.0±0.7**
50	3.9%	13.6±0.2	8.2±0.2	35.7±0.2	40.7±0.2	40.8±0.3	**43.0±0.3**	16.5±5.9	22.0±1.7	58.4±0.1	58.5±0.2	58.6±0.3	**60.5±0.2**	53.1±0.2	54.5±0.1	**58.4±0.1**	51.9±0.1	52.2±0.5	52.5±0.2
Full	100%	69.0±0.0						81.1±0.0						77.7±0.0					



(a) MiniMax

(b) Ours

Fig. 3. Failure cases of using MiniMax to fine-tune ImageNet1K. In both MiniMax and our method, the sampled categories are consomme, hotpot, trifle, ice-cream, ice lolly, French loaf, bagel, pretzel, cheeseburger, and hotdog.

ImageNet100 where MiniMax also collapsed in ImageNet100 and DiT is used. Unlike MiniMax, our method processes all 1,000 classes simultaneously, requiring only one generative model (2.5 GB) on disk. As shown in Table X, we conducted distillation experiments with various IPC values, evaluating performance on VGG11, ViT, and EfficientNet architectures. Architecture details follow the same settings as in Table VII. Since many state-of-the-art methods utilize soft labels on ImageNet1K, significantly boosting performance, we also employed ResNet18 as the teacher model and applied Kullback-Leibler divergence to train the student network. Our method consistently achieves state-of-the-art performance in most cases, further validating its effectiveness. Specifically, our method demonstrates a more significant performance improvement over the baseline model at smaller IPC values. For instance, when IPC=2 with the EfficientNet architecture, our method outperforms DiT by 1.5%. However, the performance gain diminishes at larger IPC values, likely due to the generative capacity of the DiT model reaching its saturation point. Our method inherits the authenticity characteristics of generated images, resulting in significantly better cross-architecture performance compared to statistical matching-based methods such as SRe ^{2} L and GVBSM.

We apply our idea to the distribution matching based DD method. As illustrated in the "Ours (D)" column of Table VI, the distribution matching method often outperforms MiniMax [39]. For instance, with IPC=10 and the ResNet18 architecture, it surpasses MiniMax by 2%. Another notable advantage is the high efficiency of the secondary distillation process. When IPC=50, training for 300 epochs takes just 3.5 minutes, significantly faster than the 1 h required by the gradient matching method. However, the trade-off is that distribution matching generally underperforms compared to gradient matching. For example, with IPC=10, it achieves test accuracies that are 2.3%, 0.4%, and 1.5% lower than gradient matching on ConvNet6, ResNetAP10, and ResNet18, respectively.

As shown in Table VII, the distribution matching method exhibits better performance than gradient matching in cross-architecture evaluations when IPC is relatively large. For example, with the ViT architecture and IPC=20, applying distribution matching achieves a 1.0% improvement over the baseline,

TABLE XI
ABLATION STUDIES OF EVERY COMPONENT OF OUR PROPOSED METHOD. TO MINIMIZE RESULT VARIABILITY, WE CONDUCTED 50 ROUNDS OF TRAINING AND REPORTED THE AVERAGE ACCURACY ALONG WITH THE STANDARD DEVIATION.

Dataset	IPC	Baseline	+AC	+AC+EMA	+AC+EMA+SEDS
SVHN	1	76.6±1.8	77.4±1.6	81.1±0.6	**81.5±0.4**
	10	83.9±1.1	84.5±1.0	86.4±0.5	**86.9±0.3**
	50	85.8±0.8	86.3±0.9	87.1±0.3	**87.8±0.3**
CIFAR10	1	47.8±1.2	48.0±1.1	51.6±0.6	**51.7±0.6**
	10	65.3±1.0	65.9±0.8	68.9±0.4	**69.8±0.4**
	50	71.1±0.5	71.6±0.5	73.6±0.2	**74.1±0.3**

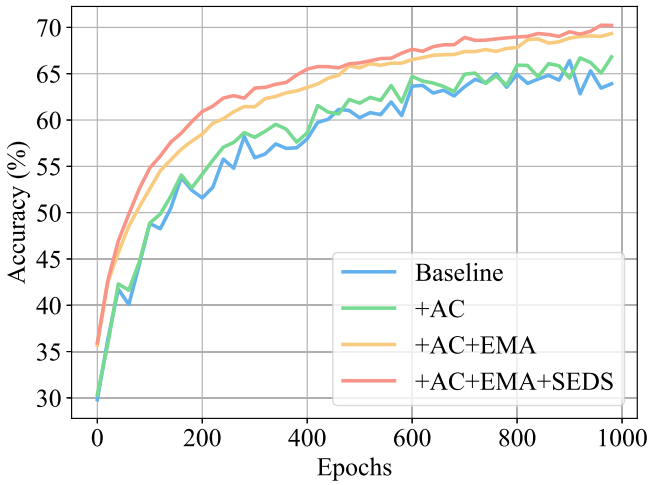


Fig. 4. Ablation study on the CIFAR10 dataset when IPC=10. At the same number of iterations, the model benefits from the EMA learning strategy, enhancing distillation performance on top of the adaptive coefficient. Finally, the inclusion of SEDS further elevates performance.

whereas gradient matching yields only a 0.1% improvement. Table X also shows the advantages of DM when applied to the large datasets.

D. Ablation Study

Table XI presents the ablation study of each proposed component in SVHN and CIFAR10 datasets where "AC" indicates adaptive coefficient, "SEDS" means Sequential Ensembling by Distillation Strategy, and we also visualize the whole validation process on the CIFAR10 dataset in Fig. 4. The efficacy of our proposed components in enhancing dataset distillation has been demonstrated.

1) Effect of the Adaptive Coefficient: The OpenTSNE [56] module is employed to depict the feature distribution of both the CIFAR10 and distilled datasets on a two-dimensional plane, as illustrated in Fig. 5. Following [57], we extract 2,048-dimensional features using a ConvNet pre-trained for 300 epochs. These features are then reduced to 2 dimensions using the t-SNE [58] algorithm. Fig. 5(a) and (c) are the worst and the best cases in the baseline method among all the seeds (seed=41 and 27, respectively). After the adaptive coefficient is adopted while conducting a matching algorithm, the generator will produce more representative samples. Specifically, as shown

TABLE XII
ABLATION STUDY OF THE PRE-TRAINED NETWORKS AND CLUSTERING WHEN DISTILLING THE SVHN AND CIFAR10 DATASETS. "AC W/O K-MEANS" REFERS TO RANDOMLY SELECTING REPRESENTATIVE SAMPLES FOR EACH CLASS TO COMPUTE DISTANCES. "3 C, 10R, 18R, 10RAP, 18RAP" REPRESENT USING 3-LAYER CONVNET, 10-LAYER RESNET, 18-LAYER RESNET, 10-LAYER RESNETAP, AND 18-LAYER RESNETAP, RESPECTIVELY.

Dataset	Pre-trained model/ Cluster	IPC		
		1	10	50
SVHN	3C	76.0±1.7	83.6±0.8	85.4±0.9
	3C-10R-18R	77.1±0.8	83.6±0.8	85.5±0.7
	3C-10R-18R-10RAP-18RAP	**77.4±1.6**	**84.5±1.0**	**86.3±0.9**
	Baseline	76.6±1.8	83.9±1.1	85.8±0.8
	AC w/o K-means	76.8±0.7	84.0±0.5	85.9±0.5
	AC w/K-means	**77.4±1.6**	**84.5±1.0**	**86.3±0.9**
CIFAR10	3C	47.3±1.0	64.8±0.8	70.7±0.4
	3C-10R-18R	47.7±1.2	65.5±0.9	71.3±0.5
	3C-10R-18R-10RAP-18RAP	**48.0±1.1**	**65.9±0.8**	**71.6±0.5**
	Baseline	47.8±1.2	65.3±1.0	71.1±0.5
	AC w/o K-means	47.6±1.1	65.7±0.8	71.4±0.5
	AC w/K-means	**48.0±1.1**	**65.9±0.8**	**71.6±0.5**

in Fig. 5(b) and (d), with the inclusion of AC, the worst-case scenario and best-case scenario obtain an improvement in accuracy by 1.36% and 0.05%, respectively. The classes circled in the figure demonstrate that with the inclusion of adaptive weights, the distilled dataset aligns more closely with the distribution of the original dataset. Furthermore, as depicted in Fig. 6(a), by comparing with the case when $\lambda_1 = 0$, we can conclude that our method is not sensitive to hyperparameters and consistently outperforms the baseline model. In addition, we demonstrate the effectiveness of the pre-trained models and clustering in the SVHN and CIFAR10 datasets in Table XII. We can conclude that 1) as the number of pre-trained architectures increases, the accuracy improves regardless of the number of distilled images per class. 2) Using randomly selected samples for adaptive coefficients outperforms the baseline at IPC=10 and IPC=50 but is less effective than using samples from the clustering algorithm, confirming the effectiveness of the clustering.

2) Effect of the Sliding Window Size: The effect of the sliding window size in the SEDS is shown in Fig. 6(b). The GPU memory required to run the program increases as the window size increases. When the window size continues to increase, the improvement in accuracy is not significant. In our experiments, we chose a window size of 10 because it strikes a balance between accuracy and memory efficiency.

3) Effect of the Hyperparameters in the Deployment Stage for High-Resolution Dataset: Fig. 6(c) shows the influence of λ_2 in (10) ranging from 0.5 to 4. In our experiments, we choose $\lambda_2 = 1$. Fig. 6(d) and (e) present the impact of the inner loop on the accuracy and the loss. As the number of inner loop iterations increases, the training time increases, and the matching loss decreases. Our experiments set the inner loop as 10 to balance accuracy and latency. Fig. 6(f) presents the influence of the epochs on the final distillation performance. As observed, convergence occurs around 300 epochs, and after that, there is a significant fluctuation in accuracy.

4) Effect of the Extended MiniMax Loss Function: As shown in Table XIII, simply extending the original MiniMax loss to

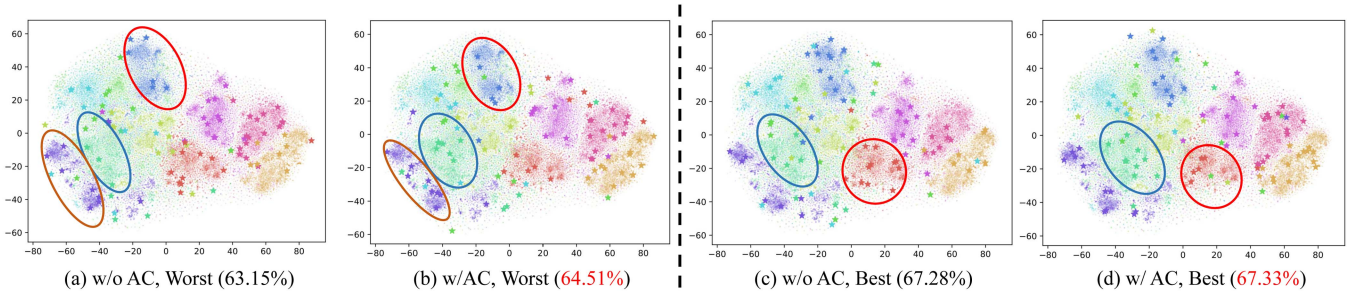


Fig. 5. Visual representation of the feature distribution of the original and distilled datasets on a two-dimensional plane. Each circle corresponds to an image from the original dataset, while the pentagon represents an image from the distilled dataset. Different colors signify distinct classes. The best and the worst refer to the highest and the lowest classification accuracies achieved by training ConvNet3 architecture on different generated images among 50 random seeds ranging from 0 to 49. (a) and (c) denote scenarios without adaptive coefficient (AC), while (b) and (d) denote scenarios with AC. A more uniform distribution of pentagons within the circles is desirable for a given class. All experiments were conducted using the CIFAR10 dataset with IPC set to 10. Note that, similar to DiM [7], our inference process involves a total output of $(IPC \times c = 10 \times 10 = 100)$ images per forward pass. Each iteration generates 100 images randomly distributed among classes, with an average of 10 images per class. Parenthesized values indicate classification accuracy on the distilled dataset. These accuracies reflect overall training results after 1,000 iterations (the same setting as DiM [7]). Therefore, $1,000 \times 100$ images are produced in total. Each subgraph displays the results of one certain iteration. Notably, AC improves alignment with class clusters, boosting classification accuracy. This underscores the efficacy of AC. Zoom in for a clearer view.

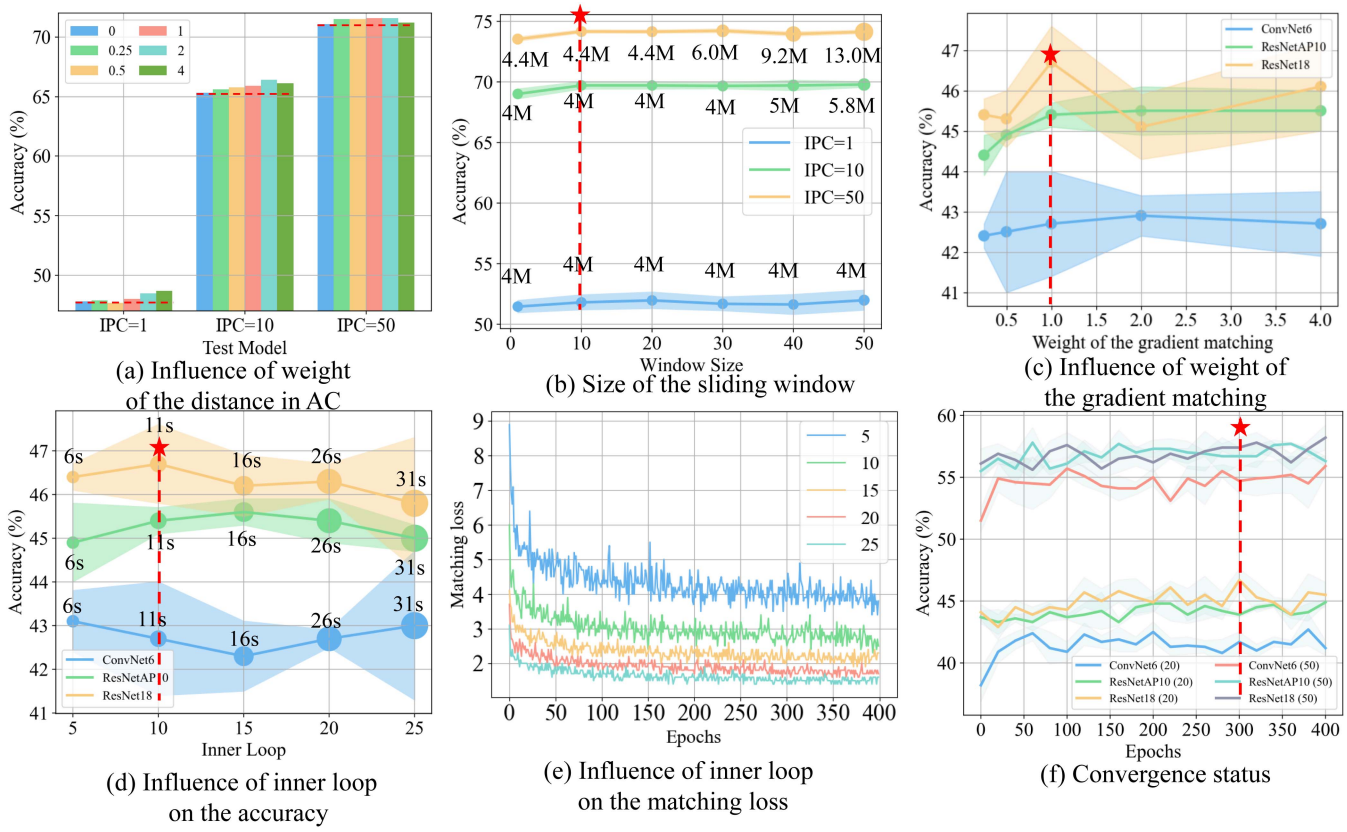


Fig. 6. Hyperparameters analysis in the proposed approach. (a) Influence of the λ_1 in (4). (b) The influence of the sliding window size in SEDS conducting on the CIFAR10 dataset. "M" represents the amount of GPU memory required to run the program. The dashed line indicates the best setting in the experiments. The shaded area represents the standard deviation across three trial results. (c) Influence of λ_2 in (10). (d)–(e) The impact of the inner loop on the accuracy and the loss. We recorded the time taken to compress latent vectors using the ConvNet6 architecture in one iteration, as well as the accuracy during training the classification model and testing with ConvNet6, ResNetAP10 and ResNet18 in (d). (f) The convergence status during the deployment stage. Experiments in (c)–(f) are conducted on the ImageWoof dataset with IPC=20.

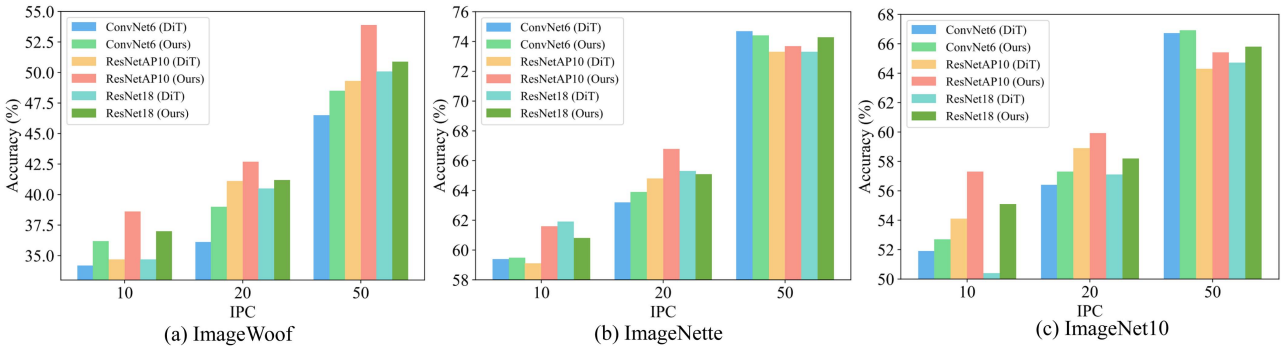


Fig. 7. Performance of DD on the high-resolution dataset without finetuning the diffusion model. In most cases, our method exhibits a noticeable improvement in performance compared to the pre-trained diffusion model.

TABLE XIII
ABLATION STUDIES OF THE DISILLATION STAGE FOR THE IMAGEWOOF AND IMAGENETTE DATASETS. THE EVALUATION ARCHITECTURE IS RESNETAP10. L_r AND L_d REPRESENT THE ORIGINAL MIN-MAX LOSSES WHILE L_r (OURS) AND L_d (OURS) REPRESENT OUR PROPOSED ONES IN (6) AND (7), RESPECTIVELY

Dataset	IPC	$+L_r$	$+L_r$ (Ours)	$+L_r+L_d$	$+L_r+L_d$ (Ours)
ImageWoof	10	35.3±0.7	36.5±0.2	**39.2±1.3**	36.7±0.3
	20	43.6±1.2	43.5±0.8	45.8±0.5	**46.2±0.4**
	50	52.8±0.2	52.8±0.5	56.3±1.0	**60.1±1.7**
ImageNette	10	61.1±1.2	61.5±1.1	**62.0±0.2**	61.8±1.1
	20	63.9±1.0	64.3±1.1	67.9±1.1	**68.1±0.4**
	50	73.5±0.8	73.0±0.8	74.8±1.1	**75.4±0.9**

TABLE XIV
EFFECT OF THE DIFFERENT KS OF OURS$*$. THE RESULTS ARE OBTAINED ON THE IMAGEWOOF DATASET

K	IPC=10			IPC=50		
	Conv6	ResNetAP10	ResNet18	Conv6	ResNetAP10	ResNet18
3	34.2±1.1	39.8±0.2	38.5±0.9	54.4±0.2	55.5±0.5	55.7±0.9
5	33.7±1.9	36.5±0.9	36.3±0.8	54.7±0.8	57.4±0.9	56.9±0.4
7	**37.2±1.7**	**39.9±1.2**	**41.1±1.0**	**56.0±1.5**	**59.3±1.0**	**61.2±1.9**
9	34.0±0.9	34.3±0.2	35.8±1.0	51.4±0.3	56.5±0.7	57.0±0.3
11	32.7±1.0	35.5±0.5	35.9±0.8	54.3±0.2	58.7±0.2	59.4±1.0

include other non-extreme samples indeed improves the distillation performance of the fine-tuned diffusion model, and the performance reaches its optimum with the incorporation of L_r (Ours) and L_d (Ours). In addition, experimental results indicate that the proposed improvement introduces negligible training overhead. On ImageWoof, both MiniMax$*$ and MiniMax require approximately 0.8 hours of training on a single RTX 4090. Table XIV further investigates the influence of different Ks in our extended MiniMax loss function. When K takes a small value, the proposed formulation essentially reduces to the original MiniMax objective. As K increases, however, the constraint on the generated images becomes less restrictive. We choose $K = 7$ for our default setting.

5) Effect of the Secondary Distillation for High-Resolution Datasets: Fig. 7 presents the distillation result without fine-tuning the pre-trained diffusion model with the MiniMax loss function on the high-resolution datasets. We can conclude that the performance of the original DiT can be further boosted with the help of secondary distillation in most scenarios. For example, when IPC=10, our method (without MiniMax loss function) surpasses DiT by 0.8%, 3.2%, and 4.7% for ConvNet6, ResNetAP10 and ResNet18 architectures, respectively, on the ImageNet10 dataset. Note that some of our results even surpass the MiniMax diffusion. For instance, when IPC=10 on the ImageNet10 dataset, our method (without MiniMax loss function) surpasses MiniMax by 1.1%, 1.4%, and 6.4% for ConvNet6, ResNetAP10 and ResNet18 architectures, respectively. The effectiveness of secondary distillation lies in its ability to fully exploit the strong generative capacity of the original DiT model. By first generating a large set of samples and then performing distillation, the process enforces the distilled images to be representative of the generated images.

6) Effect of the Original Dataset Size: The effect of the original dataset size is shown in Fig. 8. In Fig. 8, we can conclude that the performance of the final distilled dataset does not necessarily improve with an increase in the size of the dataset. For example, in the case of IPC=10/20, when the original dataset size is 1,500, the performance is worse compared to 750 and 1,000. The reasons are twofold: First, the original ImageWoof dataset has approximately 1,300 images per class. Generating 1,500 images per class using a generative model would inevitably introduce redundancy in the latent space. Second, assuming that the total distribution of generated images is fixed, as the number of generated images increases, each latent vector for distillation becomes more concentrated, leading to a lack of diversity and, consequently, a deterioration in the final results. From Fig. 9, we can observe that as the IPC of the final distilled images increases, the matching loss decreases at first because the generative model has not yet reached saturation, and more generated images contain more effective information. However, once the number of proxy images per class exceeds 1,000, the capacity of the generative model becomes saturated, and further increasing the number of generated images leads to a rise in the matching loss instead.

E. Efficiency Analysis

Using the ImageWoof and ImageNet1K datasets as representative examples, we analyze the efficiency bottlenecks of our method in terms of GPU memory usage, disk storage overhead,

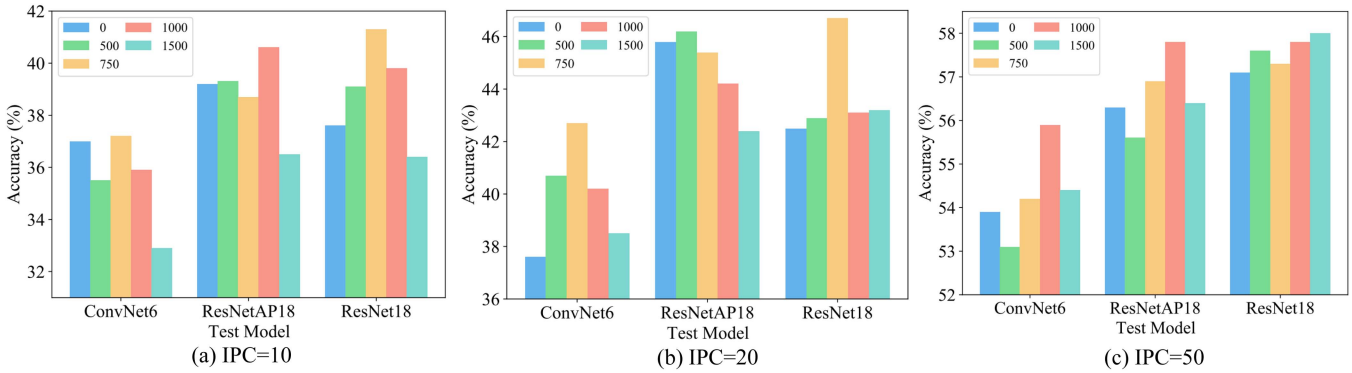


Fig. 8. The influence of the size of the original dataset on the overall accuracy of different architectures trained on the distilled dataset. All the experiments are conducted on the ImageWoof dataset. The numbers in the legend represent the sizes of the original datasets, with 0 indicating the baseline model.

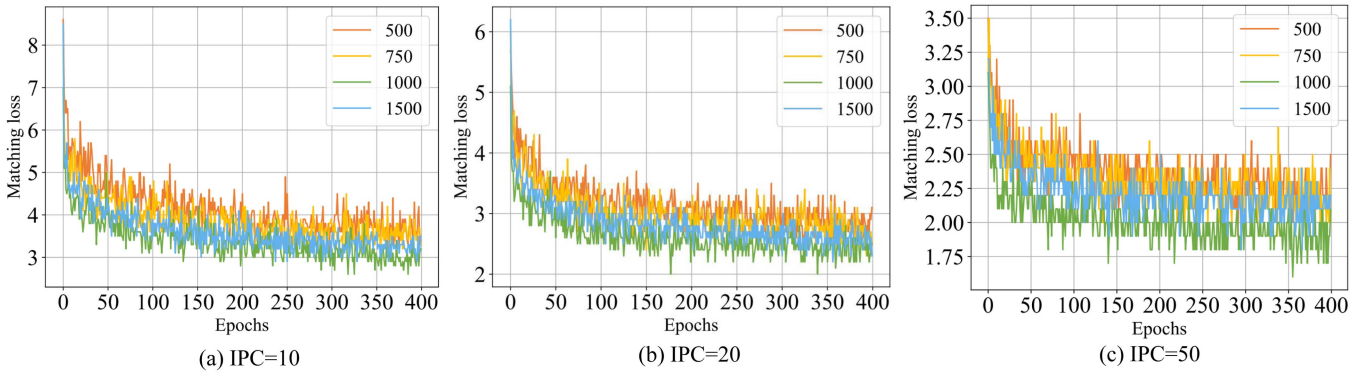


Fig. 9. The influence of the size of the original dataset on the matching loss during condensing the original dataset. All the experiments are conducted on the ImageWoof dataset. In general, the smaller the IPC, the greater the matching loss. Moreover, as the size of the proxy dataset increases, the matching loss first decreases and then increases. This implies that the reference to the matching loss can be used to determine whether the proxy dataset size can be further increased. Zoom in for the best view.

and execution time. All computations are carried out on a single NVIDIA GeForce RTX 4090 GPU.

For the ImageWoof dataset, fine-tuning the DiT model initially requires 11.4 GB of GPU memory, followed by 3.6 GB for sampling latent vectors. The subsequent stages of pretraining, compression, and decoding consume 1.2 GB, 1.0 GB, and 2.2 GB of GPU memory, respectively. In terms of disk usage, the generative model occupies 2.5 GB, while 1,000 latent vectors require 158.3 MB. Regarding time consumption, fine-tuning the DiT model takes approximately 40 minutes, and sampling 1,000 latent vectors per class requires about 2.8 hours. Notably, both fine-tuning and sampling are performed only once and can be reused across different IPC settings. Finally, distilling the latent vectors into datasets with 1, 10, and 50 IPCs takes around 0.8 hours, 0.9 hours, and 1.2 hours, respectively.

For the ImageNet1K dataset, sampling 750 latent vectors per class using the DiT model requires 11.6 GB of disk space and takes approximately 8.8 days. Similar to the ImageWoof dataset, the sampled latent vectors can be reused across different IPC settings. Finally, distilling the latent vectors into datasets with 1, 10, and 50 IPCs takes around 1 d with 12.54 GB of GPU memory, 1.1 days with 12.83 GB, and 1.25 days with 15.59 GB, respectively.

V. Conclusion and Future Work

In this paper, we propose a noise-unconstrained GM-based DD algorithm. During GAN fine-tuning, we introduce an adaptive coefficient in the matching function to enable GM to generate representative samples, where the coefficient is proportional to the distance between the cluster centers of the original and distilled images in the feature space. Moreover, the original MiniMax loss is extended to ease optimization during the fine-tuning of a diffusion model. Besides, to fully tap the potential of GM, a post-process dataset ensembling by DD strategy is proposed. We have proposed corresponding technical solutions tailored to the characteristics of GAN and diffusion models. Specifically, we introduce an online distillation approach for the fast inference speed of GAN models. In contrast, for the slower sampling speed of diffusion models, we pre-store sampled latent vectors and then perform DD in the latent space. Theoretical analysis demonstrates that the generalization error of our method decreases due to our specific design. We also explore the different processes between distilling the generated images and the original ones. This study is anticipated to advance the landscape of DD based on GM. In future work, we plan to enhance the stability of fine-tuning generative models on datasets with a large number

of categories, investigate the distillation with more advanced generative models, and examine the application of DD to more demanding computer vision tasks, including object detection and image restoration.

REFERENCES

[1] S. Lei and D. Tao, "A comprehensive survey of dataset distillation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 1, pp. 17–32, Jan. 2024.

[2] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.

[3] B. Zhao, K. R. Mopuri, and H. Bilen, "Dataset condensation with gradient matching," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–20.

[4] G. Cazenavette, T. Wang, A. Torralba, A. A. Efros, and J.-Y. Zhu, "Dataset distillation by matching training trajectories," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10708–10717.

[5] J.-H. Kim et al., "Dataset condensation via efficient synthetic-data parameterization," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 11102–11118.

[6] G. Cazenavette, T. Wang, A. Torralba, A. A. Efros, and J.-Y. Zhu, "Generalizing dataset distillation via deep generative prior," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 3739–3748.

[7] K. Wang et al., "DiM: Distilling dataset into generative model," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 42–59.

[8] D. J. Zhang et al., "Dataset condensation via generative model," 2023, *arXiv:2309.07698*.

[9] M. Mohri, A. Rostamizadeh, and A. Talwalkar, *Foundations of Machine Learning*. Cambridge, MA, USA: MIT Press, 2018.

[10] M. Kang, J. Shin, and J. Park, "StudioGAN: A taxonomy and benchmark of GANs for image synthesis," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 12, pp. 15725–15742, Dec. 2023.

[11] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville, "Improved training of wasserstein GANs," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 5767–5777.

[12] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, "Spectral normalization for generative adversarial networks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–26.

[13] A. Brock, J. Donahue, and K. Simonyan, "Large scale GAN training for high fidelity natural image synthesis," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–36.

[14] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 6840–6851.

[15] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10674–10685.

[16] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–22.

[17] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 4172–4182.

[18] E. Xie et al., "DiffFit: Unlocking transferability of large diffusion models via simple parameter-efficient fine-tuning," 2023, *arXiv:2304.06648*.

[19] O. Sener and S. Savarese, "Active learning for convolutional neural networks: A core-set approach," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–13.

[20] M. Welling, "Herding dynamical weights to learn," in *Proc. Int. Conf. Mach. Learn.*, 2009, pp. 1121–1128.

[21] T. Wang, J.-Y. Zhu, A. Torralba, and A. A. Efros, "Dataset distillation," 2018, *arXiv:1811.10959*.

[22] T. Nguyen, Z. Chen, and J. Lee, "Dataset meta-learning from kernel ridge-regression," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–25.

[23] N. Loo, R. Hasani, A. Amini, and D. Rus, "Efficient dataset distillation using random feature approximation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 13877–13891.

[24] Y. Feng, S. R. Vedantam, and J. Kempe, "Embarrassingly simple dataset distillation," in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 1–23.

[25] B. Zhao and H. Bilen, "Dataset condensation with distribution matching," in *Proc. IEEE Winter Conf. Appl. Comput. Vis.*, 2023, pp. 6503–6512.

[26] A. Sajedi, S. Khaki, E. Amjadian, L. Z. Liu, Y. A. Lawryshyn, and K. N. Plataniotis, "DataDAM: Efficient dataset distillation with attention matching," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 17051–17061.

[27] G. Zhao, G. Li, Y. Qin, and Y. Yu, "Improved distribution matching for dataset condensation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 7856–7865.

[28] W. Deng et al., "Exploiting inter-sample and inter-feature relations in dataset distillation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 17057–17066.

[29] L. Zhang et al., "Accelerating dataset distillation via model augmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 11950–11959.

[30] J. Cui, R. Wang, S. Si, and C.-J. Hsieh, "Scaling up dataset distillation to ImageNet-1 K with constant memory," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 6565–6590.

[31] Z. Yin, E. Xing, and Z. Shen, "Squeeze, recover and relabel: Dataset condensation at ImageNet scale from a new perspective," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2023, pp. 73582–73603.

[32] S. Shao, Z. Yin, M. Zhou, X. Zhang, and Z. Shen, "Generalized large-scale data condensation via various backbone and statistical matching," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 16709–16718.

[33] P. Sun, B. Shi, D. Yu, and T. Lin, "On the diversity and realism of distilled dataset: An efficient dataset distillation paradigm," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 9390–9399.

[34] Y. Liu, J. Gu, K. Wang, Z. Zhu, W. Jiang, and Y. You, "DREAM: Efficient dataset distillation by representative matching," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 17268–17278.

[35] Y. Xu et al., "Distill gold from massive ores: Bi-level data pruning towards efficient dataset distillation," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 1–10.

[36] X. Chen, Y. Yang, Z. Wang, and B. Mirzasoleiman, "Data distillation can be like vodka: Distilling more times for better quality," in *Proc. Int. Conf. Learn. Representations*, 2024, pp. 1–13.

[37] Y. Duan, J. Zhang, and L. Zhang, "Dataset distillation in latent space," 2023, *arXiv:2311.15547*.

[38] L. Li, G. Li, R. Togo, K. Maeda, T. Ogawa, and M. Haseyama, "Generative dataset distillation based on self-knowledge distillation," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2025, pp. 1–5.

[39] J. Gu et al., "Efficient dataset distillation via minimax diffusion," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 15793–15803.

[40] P. Liu and J. Du, "The evolution of dataset distillation: Toward scalable and generalizable solutions," 2025, *arXiv:2502.05673*.

[41] M. Mirza and S. Osindero, "Conditional generative adversarial nets," 2014, *arXiv:1411.1784*.

[42] M. He, S. Yang, T. Huang, and B. Zhao, "Large-scale dataset pruning with dynamic uncertainty," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, 2024, pp. 7713–7722.

[43] W. Yang, Y. Zhu, Z. Deng, and O. Russakovsky, "What is dataset distillation learning?," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 1–23.

[44] B. Zhao and H. Bilen, "Dataset condensation with differentiable siamese augmentation," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 12674–12685.

[45] K. Wang et al., "CAFE: Learning to condense dataset by aligning features," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 12186–12195.

[46] S. Liu, K. Wang, X. Yang, J. Ye, and X. Wang, "Dataset distillation via factorization," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 1100–1113.

[47] P. Sermanet, S. Chintala, and Y. LeCun, "Convolutional neural networks applied to house numbers digit classification," in *Proc. Int. Conf. Pattern Recognit.*, 2012, pp. 3288–3291.

[48] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Master's Thesis, Dept. Comput. Sci., Univ. Toronto, Toronto, ON, Canada, 2009.

[49] Y. Le and X. Yang, "Tiny ImageNet visual recognition challenge," *CS 231 N*, vol. 7, no. 7, pp. 1–6, 2015.

[50] J. Howard, "A smaller subset of 10 easily classified classes from ImageNet, and a little more french," 2019. [Online]. Available: https://github.com/fastai/imagenette

[51] Y. Tian, D. Krishnan, and P. Isola, "Contrastive multiview coding," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 776–794.

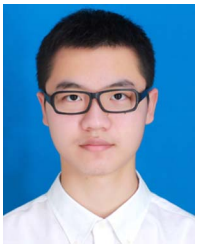
[52] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 8026–8037.

[53] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Representations*, 2015, pp. 1–14.

[54] A. Dosovitskiy et al., "An image is worth 16 $\times$ 16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–22.

[55] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6105–6114.

[56] P. G. Poličar, M. Stražar, and B. Zupan, "OpenTSNE: A modular python library for t-SNE dimensionality reduction and embedding," *J. Statistical Softw.*, vol. 109, no. 3, pp. 1–30, 2024.
[57] Z. Chen et al., "A comprehensive study on dataset distillation: Performance, privacy, robustness and fairness," 2023, *arXiv:2305.03355*.
[58] G. C. Linderman, M. Rachh, J. G. Hoskins, S. Steinerberger, and Y. Kluger, "Fast interpolation-based t-SNE for improved visualization of single-cell RNA-seq data," *Nature Methods*, vol. 16, no. 3, pp. 243–245, 2019.



Jingxuan Zhang received the BEng degree in computer science from the East China University of Science and Technology, Shanghai, China in 2022. He is currently working toward the PhD degree in computer science with the Department of Computer Science and Engineering, East China University of Science and Technology, Shanghai, China. His current research interests include dataset distillation, computer vision, and machine learning.

Lei Dai (Member, IEEE) received the BSc degree in mathematics and the MSc degree in statistics from the Huazhong University of Science and Technology, Wuhan, China, in 2015 and 2018, respectively, and the PhD degree in computer science from the University of Macau, Macau, China, in 2022. She is currently a lecturer with the Department of Computer Science and Engineering, East China University of Science and Technology, Shanghai, China. Her research interests include computer vision, image processing, and machine learning.

Fei Ye received the bachelor degree from the Chengdu University of Technology, China, in 2014, the master degree in computer science and technology from Southwest Jiaotong University, China, in 2018, and the PhD degree in computer science from the University of York, in 2024. He was a postdoctoral fellow with the Mohamed bin Zayed University of Artificial Intelligence. He is currently working on University of Electronic Science and Technology of China. His research topics include deep generative image models, lifelong learning and mixture models.

Zhihua Chen (Member, IEEE) received the PhD degree in computer science from Shanghai Jiao Tong University, Shanghai, China, in 2006. He is currently a full professor with the Department of Computer Science and Engineering, East China University of Science and Technology, Shanghai, China. His current research interests include neural network design, deep learning, and computer vision.

Ping Li (Member, IEEE) received the PhD degree in computer science and engineering from the Chinese University of Hong Kong, Hong Kong, in 2013. He is currently an assistant professor with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong. He has published more than 300 top-tier scholarly research articles, pioneered several new research directions, and made a series of landmark contributions in his areas. He has an excellent research project reported by the *ACM TechNews*, which only reports the top breakthrough news in computer science worldwide. More importantly, however, many of his research outcomes have strong impacts to research fields, addressing societal needs and contributed tremendously to the people concerned. His current research interests include AIGC, image/video generation, stylization, colorization, artistic rendering and synthesis, realism in non-photorealistic rendering, computational art, and creative media. He is a Fellow of the Computer Graphics Society. He is an associate editor of the *IEEE Transactions on Pattern Analysis and Machine Intelligence*.

Xiaokang Yang (Fellow, IEEE) received the BS degree from Xiamen University, Xiamen, China, in 1994, the MS degree from the Chinese Academy of Sciences, Shanghai, China, in 1997, and the PhD degree from Shanghai Jiao Tong University, Shanghai, in 2000. He is currently a distinguished professor with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University. From 2000 to 2002, he was a research fellow with the Centre for Signal Processing, Nanyang Technological University, Singapore. From 2002 to 2004, he was a research scientist with the Institute for Infocomm Research, Singapore. From 2007 to 2008, he visited the Institute for Computer Science, University of Freiburg, Freiburg im Breisgau, Germany, as an Alexander von Humboldt research fellow. He has published more than 200 refereed articles. He has filed 60 patents. His current research interests include image processing and communication, computer vision, and machine learning. He is a member of Asia Pacific Signal and Information Processing Association, the VSPC Technical Committee, the IEEE Circuits and Systems Society, and the MMSP Technical Committee, and the IEEE Signal Processing Society. He is an associate editor of the *IEEE Transactions on Multimedia* and a senior associate editor of the *IEEE Signal Processing Letters*. He was a series editor of Springer CCIS and an Editorial Board Member of Digital Signal Processing. He is also the Chair of the Multimedia Big Data Interest Group, MMTC Technical Committee, and the IEEE Communication Society.

Bin Sheng (Senior Member, IEEE) received the BA degree in english and the BEng degree in computer science from the Huazhong University of Science and Technology, Wuhan, China, in 2004, and the MSc degree in software engineering from the University of Macau, Taipa, Macau, in 2007, and the PhD degree in computer science and engineering from the Chinese University of Hong Kong, Shatin, Hong Kong, in 2011. He is currently a full professor with the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai, China. He is an associate editor of the Virtual Reality & Intelligent Hardware (VRIH), and managing editor of The Visual Computer, and received the 2023 Outstanding Contribution Award from the Computer Graphics Society. His current research interests include virtual reality and computer graphics.